DOCUMENTS

GENTABLE (INVOICE PLUGIN)
Administration und Konfiguration

VERSION 2.7

© Copyright 2024 otris software AG. Alle Rechte vorbehalten.

Weitergabe und Vervielfältigung dieser Publikation oder von Teilen daraus sind, zu welchem Zweck und in welcher Form auch immer, ohne die ausdrückliche schriftliche Genehmigung durch die otris software AG nicht gestattet. In dieser Publikation enthaltene Informationen können ohne vorherige Ankündigung geändert werden.

Alle in dieser Publikation aufgeführten Wort- und Bildmarken sind Eigentum der entsprechenden Hersteller.

Änderungen in der Software sind vorbehalten. Die in diesem Handbuch enthaltenen Informationen stellen keinerlei Verpflichtung seitens des Verkäufers dar.

Dokumentenversionen			
Version	**Datum**	**Bearbeiter**	**Bemerkungen / Änderungen**
V 1.0	04.07.2016	Tromm	Umstellung auf DOCUMENTS 5
V 1.1	06.10.2017	Härtel	Neue Features hinzugefügt
V 1.2	22.01.2019	Pajonzek	Überarbeitung des Handbuchs, insbesondere der Beispiele
V 2.0	25.10.2019	Pajonzek	Grundlegende Überarbeitung des GenTable - Handbuchs
V 2.1	10.12.2019	Pajonzek	Überarbeitung des Kapitels für Autocomplete
V 2.2	25.02.2020	Pajonzek	Beschreibung des globalen Events auf Tabellenebene “lastTab”.
V 2.3	27.04.2020	Pajonzek	Datenbankverbindungen mit TLS
V 2.4	23.04.2021	Pajonzek	kleinere Überarbeitungen und Ergänzungen in verschiedenen Kapiteln neue Einstellung maxHeight Ergänzungen im Kapitel 5: Troubleshooting: Fehler und Ursachen
V 2.5	23.03.2023	Pajonzek	Neuerungen für den `<option>` - Tag in Kapitel 2.4.3
V 2.6	13.04.2023	Pajonzek	Neuerungen für den `<default>` - Tag in Kapitel 2.4.3. Kapitel 2.4.2: globaler Schalter `<languagePropertiesFile>`
V 2.7	03.07.2024	Pajonzek	Gentable – Definition als globale Eigenschaft

Inhalt

1 Einleitung 4

1.1 Überblick 4
1.2 Die wichtigsten Eigenschaften & Features 4
1.3 Einsatz als Invoice-Plugin 5

2 Konfiguration & Anpassung 6

2.1 Aktivierung der GenTable - Funktionalität 6
2.2 Die globale Eigenschaft *precisionNumeric* 9
2.3 Die globale Eigenschaft *groupingNumeric* 10
2.4 Definition der Konfigurationsdatei 10
2.4.1 Schema des Aufbaus einer Konfigurationsdatei 11
2.4.2 Die Grundeinstellungen und globalen Einstellungen der Konfigurationsdatei 11
2.4.3 Die Definition der einzelnen Tabellenspalten/ -felder 15
2.4.4 Definition von Toolbar- Buttons 18
2.4.5 Die Definition eines Skriptes für Autocomplete 19
2.4.6 Die Definition eines Skriptes zum Befüllen eines SELECT – Feldes 22
2.4.7 Einschränkungen des Inhalts und der Sichtbarkeit 23
2.4.7.1 Einschränkungen des Inhalts 23
2.4.7.2 Einschränkungen der Sichtbarkeit 23
2.4.7.3 Beispieldefinitionsdatei 26
2.4.8 Abfrage externer Datenbanken 28
2.4.9 Alternative Auswahl der Konfigurationseinstellungen 30
2.4.9.1 Mehrere Konfigurationsdateien für einen Mappentypen 30
2.4.9.2 Skript zur Rückgabe des Inhalts einer Konfigurationsdatei 31
2.4.9.3 GenTable – Definition als globale Eigenschaft 32
2.5 Anpassung der benutzerdefinierten Skript-Datei 34
2.6 Internationalisierung 34
2.6.1 Sprachdateien 34
2.6.2 Konfigurationsdatei 35

3 Implementierung eigener Funktionalität 36

3.1 Einführung 36
3.2 Registrierung und Callback-Funktionen 36
3.2.1 Callback-Funktionen für Buttons 36
3.2.2 GentableContext 37
3.2.3 GentableGridModel 37
3.2.4 GentableGridColumnModel 39
3.2.5 GentableGridRowModel 39

4 Einsatzbeispiele & Standardfunktionen 41

5 Troubleshooting: Fehler & Ursachen 45

6 Abbildungsverzeichnis 48

1 Einleitung

1.1 Überblick

GenTable eignet sich generell dafür, beliebige tabellarische bzw. positionsbezogene Daten auf benutzerdefinierte Weise darzustellen. Diese Daten können daraufhin durch den Anwender bearbeitet und gespeichert werden.

Weiterhin ist die Integration selbst implementierter Funktionalität möglich. Auch externe Daten aus relationalen Datenbanken lassen sich einbinden.

Wichtige Merkmale sind darüber hinaus die Speicherung der Tabellendaten und der Konfigurationsdaten im XML-Format sowie die clientseitige Ausführung eines Großteils der Anwendung.

Die webbasierte Software-Komponente GenTable ist bereits in **DOCUMENTS 5** integriert und muss nicht separat installiert werden. Sofern eine gültige Lizenz vorhanden ist, kann GenTable direkt verwendet werden.

1.2 Die wichtigsten Eigenschaften & Features

Die wichtigsten Eigenschaften und Features der GenTable-Komponente:

- benutzerdefinierte Darstellung von beliebigen tabellarischen bzw. positionsbezogenen Daten
- Speicherung der Tabellendaten und der Konfigurationsdaten im *XML*-Format
- clientseitige Bearbeitung der Daten
- mögliche Elementtypen der Tabellenausgabe sind einzeilige und mehrzeilige Textfelder, Klapplisten, Kontrollkästchen (Checkboxen), statischer Text und Buttons
- anwendungsspezifischere Hilfs-Funktionen, z.B. Splitbuchung sowie automatisches Berechnen einzelner Positionsbeträge und des Gesamtbetrags bei Rechnungsmappen
- Erweiterbarkeit durch Integration eigener Funktionalität mit dem Client SDK
- benutzerdefinierte Integration von beliebigen externen Daten aus relationalen Datenbanken in ein- und mehrzeilige Textfelder, Klapplisten etc.
- externe Daten können zusätzlich vom eingeloggten Benutzer, von beliebigen Werten der Mappe oder von Werten derselben Zeile abhängig sein
- Tabellenzeilen können abhängig von der Definition sichtbar, unsichtbar oder nicht beschreibbar (readonly) sein
- automatisch ausgeführte Ereignisse mit bestimmten Tabellenspalten /-feldern verknüpfen.

1.3 Einsatz als Invoice-Plugin

Das bisher bekannteste Einsatzbeispiel von GenTable ist die Verwendung für die Eingangsrechnungsprüfung in Rechnungsmappen als sogenanntes **Invoice-Plugin**. Die Tabellenspalten entsprechen in diesem Fall den einzelnen Kategorien einer Rechnung (z.B. Bestellnummer, Menge, Einzelpreis, Gesamtpreis, Kostenstelle etc.) und die Tabellenzeilen entsprechen den einzelnen Rechnungspositionen.

2 Konfiguration & Anpassung

2.1 Aktivierung der GenTable - Funktionalität

Um GenTable in **DOCUMENTS 5** zu konfigurieren, muss zunächst der **DOCUMENTS-Manager** gestartet werden. In der Baumstruktur im linken Teil des Hauptfensters werden die einzelnen vorhandenen Mappentypen unter den Punkten *Documents* → *Mappentypen* aufgelistet.

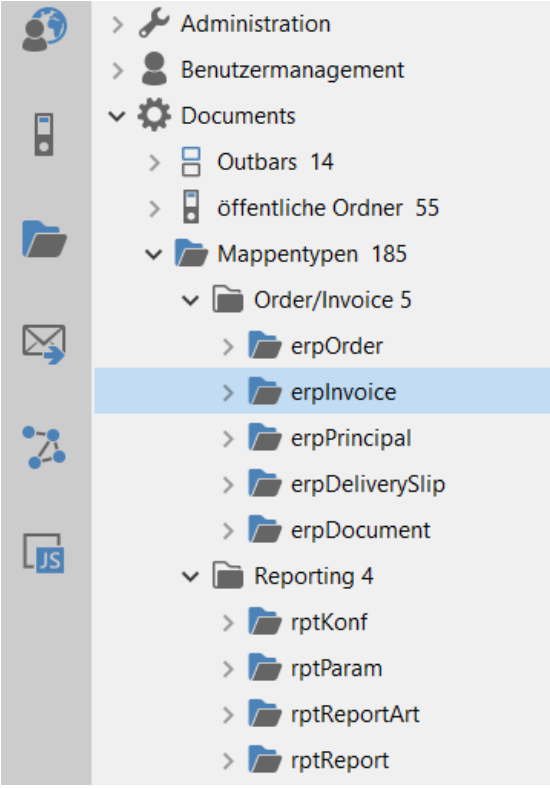


Abb. 1: Baumstruktur im DOCUMENTS-Manager

Nun öffnet man entweder einen der vorhandenen Mappentypen durch Doppelklick auf den jeweiligen Mappentyp-Eintrag in der Baumstruktur, oder man erstellt einen neuen Mappentyp über den Button *Neu* links oben in der Menübar.

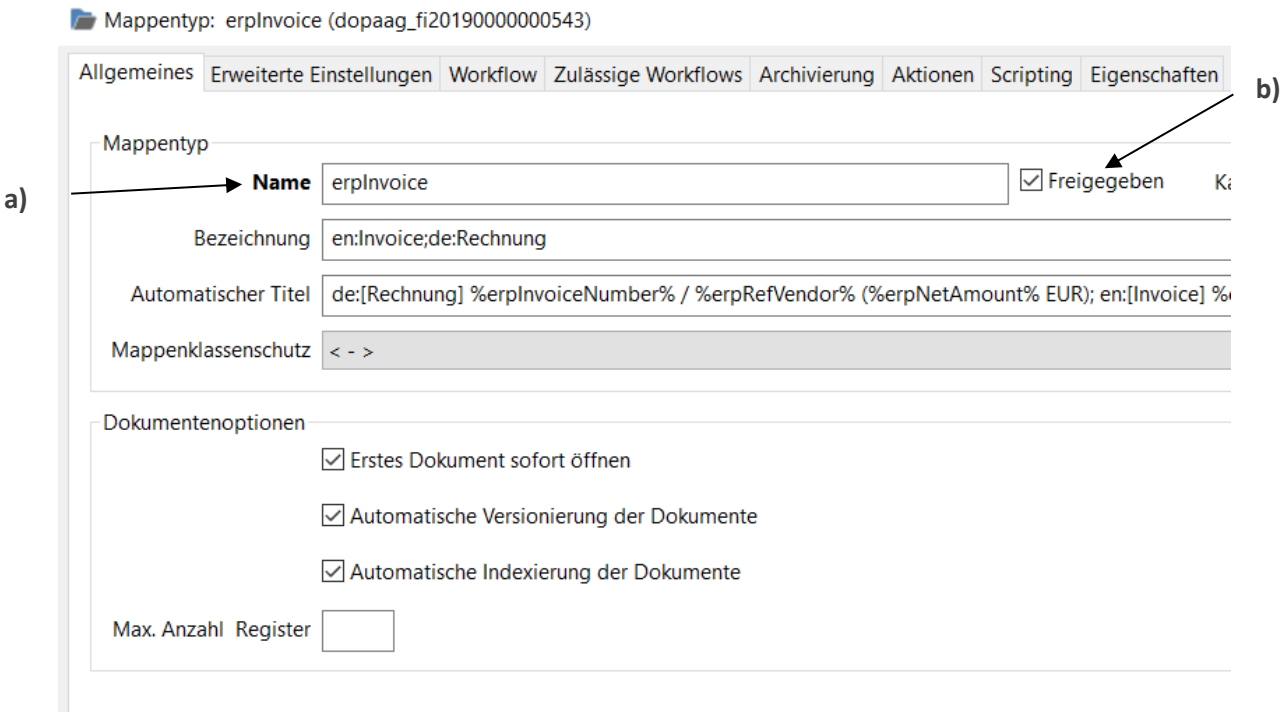


Abb. 2: Bearbeiten bzw. Erstellen eines Mappentyps

Wichtige Einstellungen bei einem Mappentyp sind die beiden folgenden Punkte in der Kategorie *Allgemeines*:

- **a)** Im Textfeld *Name* muss der eindeutige Name des Mappentyps angegeben werden
- **b)** Das Häkchen bei der Checkbox *Freigegeben* muss gesetzt sein, um den Mappentyp in **DOCUMENTS 5** freizugeben

Um GenTable in den Mappentyp zu integrieren, muss folgende Einstellung in der Kategorie *Eigenschaften* vorgenommen werden:

Über einen Rechtsklick in diesem Fenster und die dann folgende Auswahl der Funktion *neue Eigenschaft hinzufügen* muss eine neue Eigenschaft mit der Bezeichnung `hasInvoicePlugin` mit dem Wert `true` erstellt werden.

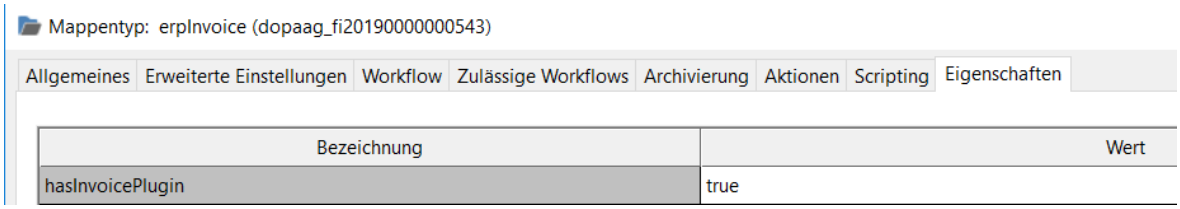
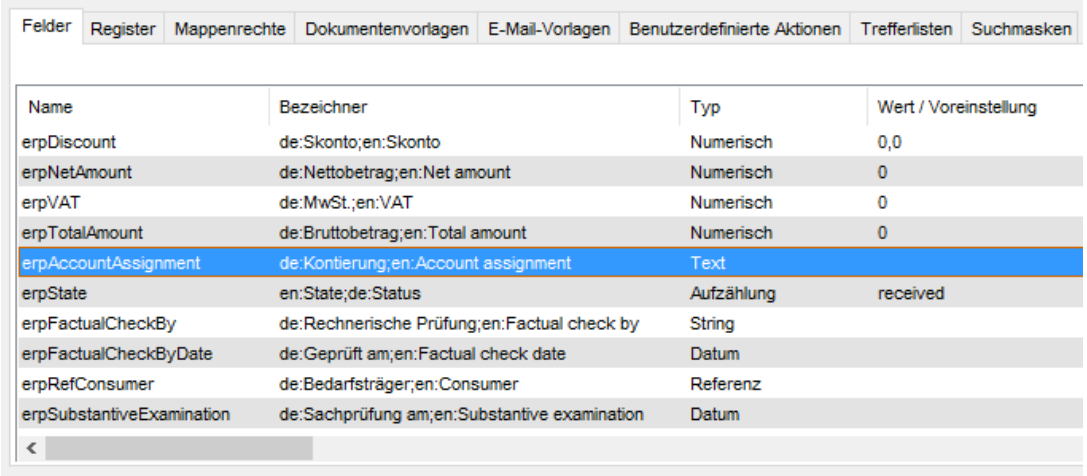


Abb. 3: Eigenschaft *hasInvoicePlugin* am Mappentyp erpInvoice

Im unteren Teil des Fensters zur Bearbeitung des Mappentyps sind in der Kategorie *Felder* die einzelnen Felder des Mappentyps aufgelistet. Die vorhandenen Felder können hier durch einen Doppelklick auf den jeweiligen Eintrag bearbeitet werden. Durch einen Rechtsklick in diesem Teil des Fensters und Auswahl der Funktion *Neuen Datensatz anlegen und in Felder einfügen* können weitere Felder erstellt werden.



Name	Bezeichner	Typ	Wert / Voreinstellung
erpDiscount	de:Skonto;en:Skonto	Numerisch	0,0
erpNetAmount	de:Nettobetrag;en:Net amount	Numerisch	0
erpVAT	de:MwSt.;en:VAT	Numerisch	0
erpTotalAmount	de:Bruttobetrag;en:Total amount	Numerisch	0
erpAccountAssignment	de:Kontierung;en:Account assignment	Text	
erpState	en:State;de:Status	Aufzählung	received
erpFactualCheckBy	de:Rechnerische Prüfung;en:Factual check by	String	
erpFactualCheckByDate	de:Geprüft am;en:Factual check date	Datum	
erpRefConsumer	de:Bedarfsträger;en:Consumer	Referenz	
erpSubstantiveExamination	de:Sachprüfung am;en:Substantive examination	Datum	

Abb. 4: Bearbeiten bzw. Erstellen von Feldern

Für GenTable ist es erforderlich, dass ein zusätzliches, nicht-sichtbares Feld vom Typ „Text“ für die Speicherung der XML-Tabellendaten existiert bzw. erstellt wird. In diesem Beispiel ist dies das Feld *erpAccountAssignment*. Dieses Feld nennt man auch Kontierungsfeld.

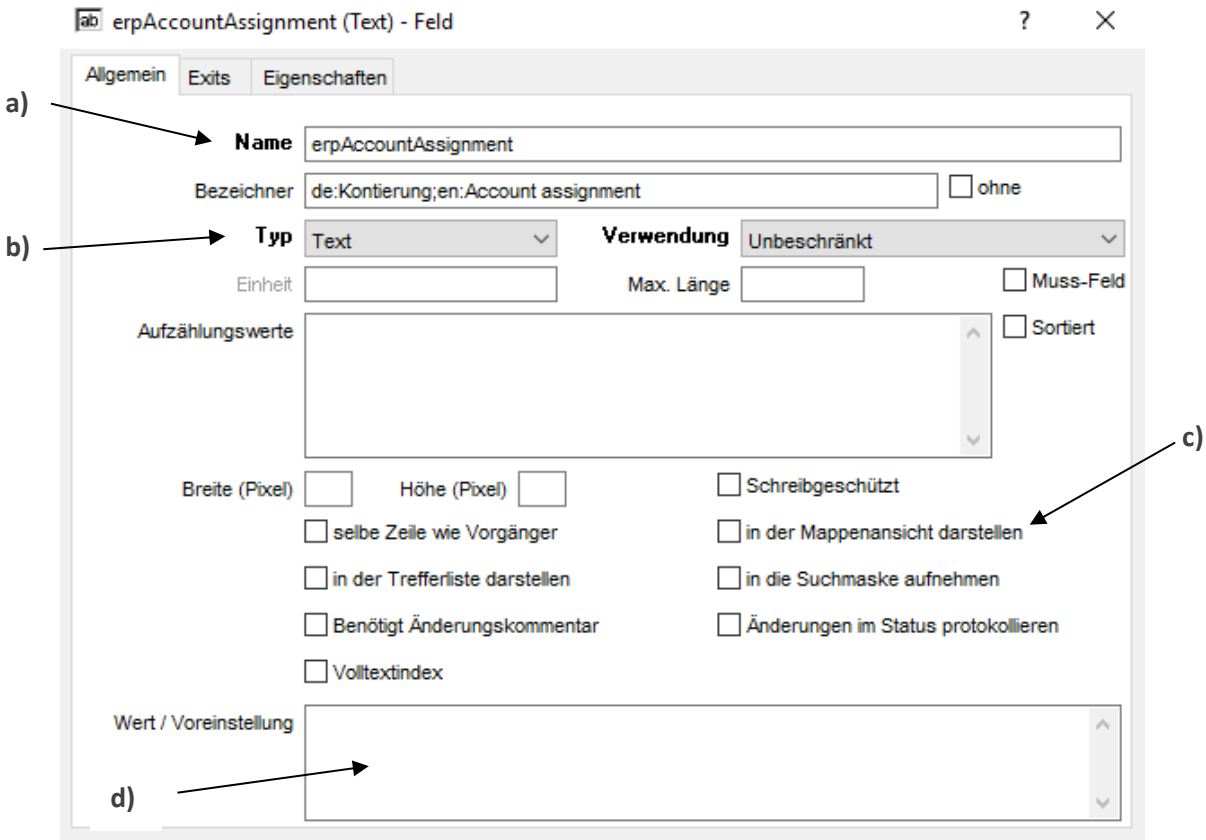


Abb. 5: Mappenfeld-Dialog

Die wichtigsten Punkte bei der Konfiguration des unsichtbaren Feldes für die XML-Tabellendaten sind die folgenden Einstellungen:

a) Im Textfeld *Name* muss ein eindeutiger Name für das Kontierungsfeld vergeben werden. Dieser Name muss später auch in der XML-Konfigurationsdatei von GenTable unter `<fieldName>` oder `<xmlfield>` verwendet werden.
b) Der Typ des Feldes für die Tabellendaten sollte *Text* sein.
c) Da das Feld mit den XML-Tabellendaten nicht angezeigt werden darf, wird das Häkchen bei der Checkbox *in der Mappenansicht darstellen* nicht gesetzt.
d) In das Textfenster *Wert/Voreinstellung* können optional vorab definierte XML-Tabellendaten eingetragen werden. Die hier eingetragenen Daten werden bei jedem Neuanlegen einer Mappe dieses Typs in der Tabelle von GenTable standardmäßig eingetragen. Durch einfaches Setzen der XML-Tags für Zeilen `<tr></tr>` und Spalten`<td></td>` kann so schon vorher festgelegt werden, wie viele Zeilen beim Neuanlegen automatisch angezeigt werden. Einträge von Spalten mit statischem Text oder von Spalten, die als nicht editierbar definiert sind, können so über die *Voreinstellung* gesetzt werden, da diese nicht mehr vom Benutzer vorgenommen werden können. Die Tabellendaten werden immer in folgender XML-Struktur gespeichert (hier beispielhaft eine Tabelle mit drei Spalten und drei Zeilen):

```
<table>
  <tr>
        <td title="Spalte 1">Beispieleintrag1</td>
        <td title="Spalte 2">Beispieleintrag2</td>
        <td title="Spalte 3">Beispieleintrag3</td>
  </tr>
  <tr>
        <td title="Spalte 1">Beispieleintrag4</td>
        <td title="Spalte 2">Beispieleintrag5</td>
        <td title="Spalte 3">Beispieleintrag6</td>
  </tr>
  <tr>
        <td title="Spalte 1">Beispieleintrag7</td>
        <td title="Spalte 2">Beispieleintrag8</td>
        <td title="Spalte 3">Beispieleintrag9</td>
  </tr>
</table>
```

Die grundlegenden Einstellungen zur Einbindung von GenTable in den **DOCUMENTS 5** Mappentypen sind damit abgeschlossen.

2.2 Die globale Eigenschaft *precisionNumeric*

Mit der globalen Eigenschaft precisionNumeric wird festgelegt, mit wieviel Nachkommastellen Zahlen in Mappen- oder GenTable - Feldern **gespeichert** werden. Mit -1 (Default – Wert) wird festgelegt, dass alle Zahlen so gespeichert werden, wie sie angezeigt

werden. Die Anzahl der angezeigten Nachkommastellen kann mit dem Tag <numberFormat> (vgl. Kap.: 2.4.3) definiert werden.

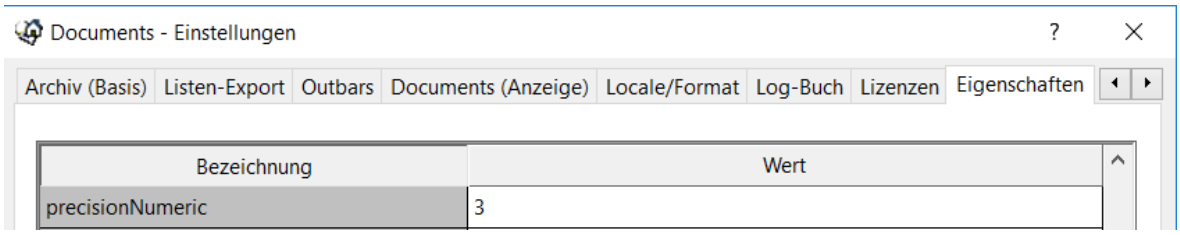


Abb. 6: Die globale Eigenschaft *precisionNumeric*

2.3 Die globale Eigenschaft *groupingNumeric*

Durch Aktivierung der globalen Eigenschaft *groupingNumeric* (Default- Wert. 1) werden alle numerischen Felder mit Tausendertrennzeichen dargestellt. Dabei können länderspezifische Trennzeichen mit der Eigenschaft *groupingNumeric_locale* deklariert werden. Für die Schweiz würde eine entsprechende, länderspezifische Auszeichnung wie folgt aussehen (*groupingNumeric_ch*='). Dabei ist zu beachten, dass man "ch" bei den Mandanten - Einstellungen unter Portal-Sprache(n) als Locale einfügt.

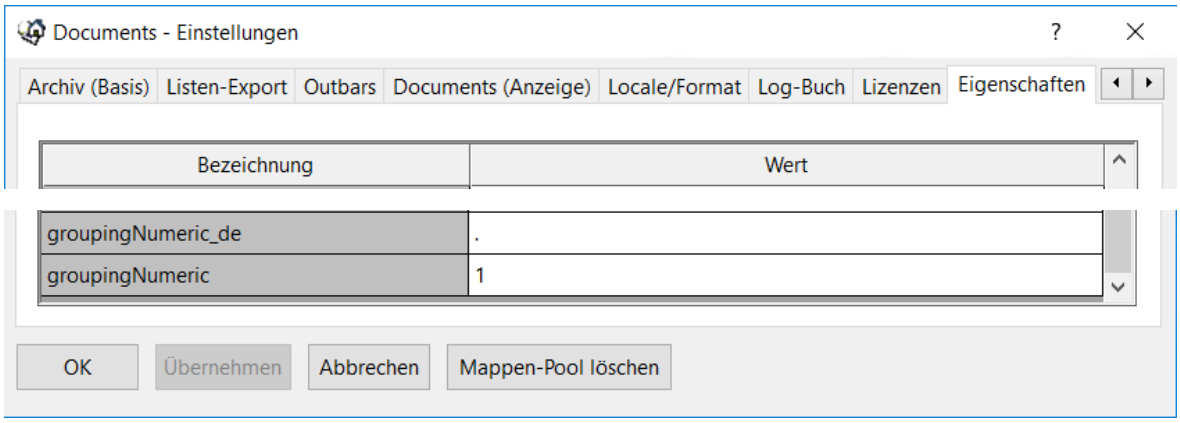


Abb. 7: Die globalen Eigenschaften *groupingNumeric und groupingNumeric_locale*

2.4 Definition der Konfigurationsdatei

Jede Konfiguration von GenTable bezieht sich in der Regel auf einen bestimmten Mappentypen in **DOCUMENTS 5**. Dazu wird im Verzeichnis `.../tomcat8/webapps/documents/web/WEB-INF/classes/` der **DOCUMENTS 5** - Installation eine XML-Konfigurationsdatei mit dem Namen `<FileTypeName>_Def.xml` hinterlegt. `<FileTypeName>` wird hier durch den technischen Namen des Mappentypen (z.B. `erpInvoice_Def.xml`) ersetzt. Als Alternative dazu kann die Konfiguration auch als sogenannte externe Ressource in **DOCUMENTS 5** eingebunden werden. - Im Folgenden werden die einzelnen XML-Elemente der Konfigurationsdatei detailliert erläutert.

2.4.1 Schema des Aufbaus einer Konfigurationsdatei

Anbei erhalten Sie einen Überblick über den Aufbau einer Konfigurationsdatei für GenTable. Die eigentliche Konfiguration ist dabei nicht vollständig, lediglich die Gliederung und der Aufbau einer solchen Datei sollen hier verdeutlicht werden. Die einzelnen Elemente werden dann ab dem Kapitel 2.4.2 beschrieben.

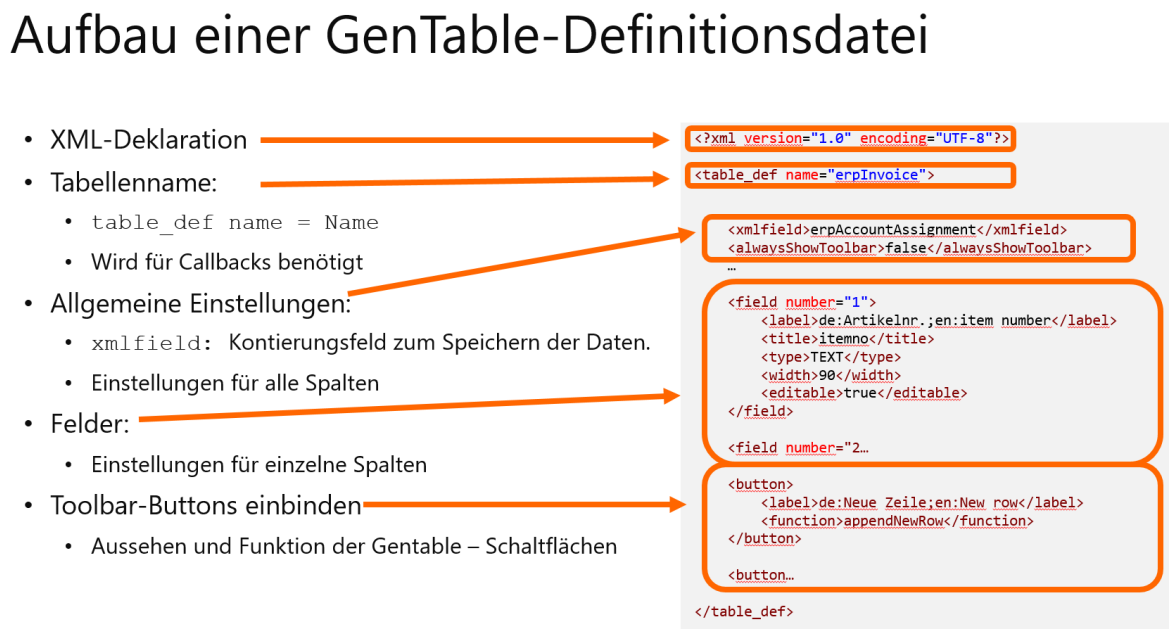


Abb. 8: Schema einer GenTable – Konfigurationsdatei

2.4.2 Die Grundeinstellungen und globalen Einstellungen der Konfigurationsdatei

`<table_def name=></table_def>`: Das Wurzelelement der Konfigurationsdatei. Das Attribut `name` enthält den eindeutigen Namen der Konfiguration. Alle weiteren Elemente werden als Kindelemente von `<table_def>` definiert.

`<fieldName></fieldName>`***:*** Der Name des Kontierungsfelds, in welchem die Daten von GenTable gespeichert werden sollen. Dieses Feld lässt sich für die Speicherformate json und xml verwenden.

`<storeFormat></storeFormat>`***:*** Mit diesem Element kann das Speicherformat festgelegt werden. Die Angabe dieses Elements ist optional. Mögliche Werte sind **xml** und **json**. Wird hier keine Einstellung vorgenommen, ist das Standardformat **xml**. Bitte beachten Sie, dass für das Speicherformat json das Kontierungsfeld auf jeden Fall mit `<fieldName>` deklariert werden sollte.

`<storeScript></storeScript>`: Skript, das zum Laden und Speichern der GenTable-Daten verwendet werden kann. Dieses Skript muss als Portalskript hinterlegt werden. Hier müssen die Daten beim Laden im definierten storeFormat (XML oder JSON) zurückgegeben werden. Beim Speichern der Daten wird der Parameter gentableData mit den zu speichernden Daten befüllt. Eine Anleitung hierzu finden Sie in der HowTo – Sammlung. (GenTable → Speichern / Laden per Skript) (vgl.: **https://otris.software/documents/howto/gentable/xmlScript.html**)

Für die beiden Elemente `fieldName` **und** `storeScript` **können auch weiterhin die Elemente** `xmlfield` **und** `xmlscript` **verwendet werden.**

Achtung: Es darf entweder `fieldName` **oder** `storeScript` in der Definitionsdatei verwendet werden. Das `storeScript` ersetzt die Speicherung mit `fieldName`.

Speicherverfahren	**Einstellung in der Konfigurationsdatei**	**Veraltete Einstellung in der Konfigurationsdatei**
Kontierungsfeld	<fieldName></fieldName>:	<xmlfield></xmlfield>:
Portalskript	<storeScript></storeScript>:	<xmlscript></xmlscript>

Abb. 9: Übersicht der Speicherverfahren

Mit dem Element `<database></database>` können die benötigten Parameter für die Verbindung zu einer Datenbank via JDBC für diese Tabelle festgelegt werden. JDBC-URL der Datenbank, JDBC-Treiber, Benutzername und Passwort können über die Unterelemente `<user> </user>`, `<password> </password>`, `<url> </url>` und `<driver> </driver>` angegeben werden.

Eine vollständige Konfiguration zum Zugriff auf eine Beispieldatenbank könnte wie folgt aussehen:

```
<database>
        <user>Benutzername</user>
        <password>Passwort</password>
        <url>jdbc:jtds:sqlserver://localhost:1433/MeineDatenbank</url>
        <driver>net.sourceforge.jtds.jdbc.Driver</driver>
  </database>
```

Bei Verwendung einer Datenbankverbindung mit TLS muss entsprechend „;ssl=require“ ergänzt werden. So entsteht die vollständige URL: <url>jdbc:jtds:sqlserver://localhost:1433/MeineDatenbank;ssl=require</url>. Es muss darüber hinaus ein TLS – fähiger Treiber verwendet werden.

`<indexNumbers></indexNumbers>`***:*** Legt mit `true` oder `false` fest, ob vor jeder Zeile eine fortlaufende Zeilennummer angezeigt werden soll. Der Default-Wert ist `true`.

`<indexNumbersColumnResizable></indexNumbersColumnResizable>`: Legt mit `true` oder `false` fest, ob die Zeilennummernspalte in ihrer Breite angepasst werden kann. Der Default-Wert ist true.

`<indexNumbersColumnWidth></indexNumbersColumnWidth>`: Legt die Standardbreite der Zeilennummernspalte in Pixeln fest (z.B. `50`).

`<indexCheckboxes></indexCheckboxes>`: Legt mit true oder false fest, ob vor jeder Zeile eine Checkbox zur Auswahl der Zeilen angezeigt werden soll. Der Default-Wert ist true.

`<lastRowEditable></lastRowEditable>`: Standardmäßig ist **bei der Neuanlage** einer Zeile **jedes Feld** editierbar. Das gilt auch für die Felder, die für das Element `<editable>` (vgl.

Kap. 2.4.3) den Wert `false` haben. Falls jedoch `<lastRowEditable>` auf den Wert `false` gesetzt wird, sind diese Felder auch bei der Neuanlage einer Zeile nicht editierbar. Der Default-Wert ist `true`. Der Wert `<editable>false</editable>` setzt sich gegenüber `<lastRowEditable>true</lastRowEditable>` durch, wenn in der Datei invoice.js *row.reload();* eingetragen ist (vgl. Kap. 3.2.1). In diesem Fall wird die letzte / neueste Zeile durch das erneute Laden mit *row.reload();* nicht mehr als neu betrachtet und daher schreibgeschützt.

`<columnsAlwaysVisible></columnsAlwaysVisible>`: Wenn die Zellen einer Spalte per Regel immer unsichtbar dargestellt werden (vgl. Kap.: 2.4.7.2), dann sieht man in der Ansicht trotzdem immer noch den Spaltennamen mit seinen Leereinträgen. Hiermit können solche Spalten komplett ausgeblendet werden. Der Default-Wert ist `true`. Er gilt wenn der Schalter nicht gesetzt ist.

`<searchable></searchable>`: Zeigt bei `true` in der Buttonzeile ein Suchfeld zum clientseitigen Filtern der angezeigten Tabellenzeilen an. Der Default-Wert ist `false`.

`<alwaysShowToolbar></alwaysShowToolbar>`: Legt mit `true` oder `false` fest, ob die Toolbar auch im Lesemodus der aktuellen Mappe angezeigt werden soll. Der Default-Wert ist `true`.

`<rowHeight></rowHeight>`: Legt die Standardhöhe einer Zeile in Pixel fest (z.B. `50`).

`<resizableRows></resizableRows>`: Legt mit `true` oder `false` fest (Default: false), ob die Zeilehöhe mit der Maus angepasst werden kann. Dies ist vor allem bei Zeilen mit großem Inhalt sinnvoll. Die manuell veränderte Zeilenhöhe wird nicht gespeichert. Sie ist nur in der aktuellen Ansicht sichtbar. Beim Speichern / Abbrechen / Aktualisieren geht sie wieder verloren.

`<enableTextSelection></enableTextSelection>`: Legt mit true oder false (Default: false) fest, ob der Text der Zeilen über mehrere Felder hinweg (auch im Lesemodus), selektiert werden kann.

`<moveRows></moveRows>` (Default: false) Erlaubt bei `true` das Verschieben von Zeilen per *Drag and Drop* im Bearbeitungsmodus der Mappe.

`<showMoveColumn></showMoveColumn>`: (Default: false) Zeigt bei `true` eine Zusatzspalte mit Anfasser zum Verschieben der jeweiligen Zeile. Wenn `enableTextSelection` und `moveRows` aktiviert sind, wird diese Spalte automatisch angezeigt.

`<quickfilter></quickfilter>`: Legt mit `true` oder `false` (Default: false) fest, ob über den einzelnen Spalten ein Suchfeld angezeigt werden soll, welches die angezeigten Zeilen nach dem Suchbegriff filtert.

`<stickyGroupRows></stickyGroupRows>`: Legt mit true oder false (Default: false) fest, ob der Header der Gruppenzeile beim Runterscrollen in der Anzeige sichtbar bleiben soll.

`<autoHeight></autoHeight>`: Legt mit true oder false (Default: false) fest, ob die Höhe des Gentables an die Anzahl der angezeigten Zeilen angepasst wird. Nützlich für Gentables, die als Gadgetfeld eingebunden werden. Wenn diese Option in einem Gentable-Gadget eingeschaltet ist, darf im Gadget nicht die Höhe mit Gadget.setStyle("height", ...) gesetzt werden, da ansonsten ein Scrollbalken erscheint.

`<maxHeight></maxHeight>`: Legt bei eingeschalteter `<autoHeight>` fest, ab welcher Höhe in Pixeln eine Scrollleiste für die Zeilen im Gentable angezeigt werden soll.

`<resizeColumnsToContent></resizeColumnsToContent>`: Legt mit `true` oder `false` (Default: false) fest, ob die Breite der Spalten an die Inhalte der Felder angepasst werden soll. Hierzu muss die Definition einer festen Spaltenbreite mit `<width></width>` ausgeschaltet sein.

`<addOverflowToAutoColumn></addOverflowToAutoColumn>`: Funktioniert nur wenn `resizeColumnsToContent=true` ist und legt mit `true` oder `false` fest, ob Spalten mit automatischer Breite um die Breite des überstehenden Inhalts nachfolgenger fester Spalten verbreitert werden sollen. Der Default ist `false`.

`<saveAll></saveAll>`: Legt mit `true` oder `false` (Default: false) fest, ob Daten, die nicht in der Definitionsdatei als Felder definiert sind, mitgespeichert werden sollen.

Wenn eine Spalte aus der GenTable – Definition entfernt wird, würden normalerweise auch die darin gespeicherten Daten in einer Mappe verloren gehen. Mit der Einstellung `true` bleiben die Daten erhalten für den Fall, dass die Spalte nachträglich wieder in der GenTable–Definitionsdatei ergänzt wird.

Dies findet in folgendem Szenario Anwendung:

1.) Daten werden im GenTable in der Spalte „Aufgaben“ o.ä. ergänzt.
2.) Spalte wird in der Definition entfernt.
3.) Es werden weitere Zeilen ohne diese Spalte ergänzt.
4.) Spalte wird in der Definition wieder hinzugefügt.
5.) Daten aus 1.) sind weiterhin vorhanden.

`<event></event>`: Registriert ein bestimmtes Ereignis (Event) für die gesamte Gentable–Tabelle. Im Moment existiert nur das Ereignis `<name>lastTab</name>`. Dieses wird ausgelöst, wenn in der letzten Zeile und gleichzeitig letzten Spalte einer Gentable – Tabelle auf die Taste „Tab“ gedrückt wird. Mit diesem Ereignis könnte zum Beispiel eine Funktion verbunden werden, die eine neue Zeile im Gentable erzeugt. Ein Beispiel für eine solche Event – Definition lautet wie folgt:

```
<event>
  <name>lastTab</name>
  <function>Funktionsname</function>
</event>
```

`<languagePropertiesFile> </languagePropertiesFile>`: Registriert eine Datei mit sprachlichen Bezeichnern. Der Dateiname muss den folgenden Aufbau haben: `Name_Sprachkürzel.properties`. Ein möglicher Dateiname für die Sprache Deutsch könnte also sein: `GentableMessages_de.properties`. Dieser wird wie folgt deklariert:

`<languagePropertiesFile>GentableMessages</languagePropertiesFile>` Die Datei muss, ebenfalls wie die Gentable – Definitionsdatei, in der externen Ressource eingebunden sein.

Die sprachlichen Bezeichner werden anhand von Schlüssel – Wert – Paaren in der Datei abgelegt. – Die Werte unterscheiden sich je nach Sprache.

Bsp.:

`unitprice=Stückpreis`

Aufgerufen werden können die Werte dann mit der pf: - Notation (vgl. Kap.: 2.4.3 und die dortige Beschreibung der `<option>` - Tags).

2.4.3 Die Definition der einzelnen Tabellenspalten/ -felder

Mit dem Element `<field></field>` wird eine Spalte der Tabelle definiert. Es muss stets **mindestens eine Tabellenspalte** definiert werden.

Die Detaileinstellungen der jeweiligen Spalte werden mit den folgenden Unterelementen festgelegt.

`<title></title>` (Pflichtfeld): Enthält den **technischen** Namen der Spalte.

`<label></label>`***:*** Enthält eine (optionale) Spaltenüberschrift, welche in der Benutzeroberfläche angezeigt wird. (kann internationalisiert werden, siehe Abschnitt 2.6.2)

`<type></type>`***:*** Datentyp des Feldes. Es stehen folgende Datentypen zur Verfügung:

- **TEXT,** einzeiliges Textfeld
- **TEXTAREA,** mehrzeiliges Textfeld für längeren Text
- **SELECT,** Klappliste mit Liste von vordefinierten Einträgen
- **CHECKBOX,** Checkbox (Auswahlkästchen zur Selektion per Häkchen)
- **STATICTEXT,** statischer, nicht-änderbarer Text. Generell sind staticText – Felder nicht editierbar. Man könnte hier jedoch über einen <default></default> - Wert sicherstellen, dass in dem Feld ein Wert zu sehen ist. Wenn <editable>true</editable> eingestellt ist, lässt sich die letzte Zeile bei der Neuanlage editieren. Dabei kann auch <lastRowEditable>false</lastRowEditable> eingestellt sein. Der Feldtyp ist zu Beginn auf readonly gestellt, ähnlich einer condition readonly – Anweisung (vgl. Kap.: 2.4.7.2). Das bedeutet, dass sich das Feld nicht beschreiben lässt, falls der vormals genannte Schalter nicht gesetzt ist.
- **BUTTON,** Schaltfläche: Um diese Schaltfläche benutzbar zu machen, muss dieses Feld explizit auf <editable>true</editable> eingestellt sein.
- **DATE2,** Datumsfeld mit Kalender
- **TIMESTAMP2,** Zeitstempel mit Kalender und Uhrzeit
- **NUMBER,** Zahl

Anmerkung:
Die veralteten Datentypen DATE und TIMESTAMP werden noch angezeigt, sollten aber nicht benutzt werden, da diese Feldtypen das Datum immer sprachabhängig speichern. Dies führt bei mehrsprachigen Installationen zu Problemen, da es hier z.B. passieren kann, dass verschiedene Felder in unterschiedlichen Sprachen bzw. Datumsformaten befüllt werden.

DATE2 und TIMESTAMP2 speichern das Datum in dem im Manager definierten Standardformat (Locale – Formateinstellungen - Datumsformat).

`<default></default>`: Ausgangswert des Feldes bei Erzeugung einer Zeile. Hier kann manuell ein Wert eingetragen werden.

Es kann aber auch mit

`%FILE_FIELD:<Feldname>%`

der aktuelle Wert eines Mappenfeldes als Ausgangswert verwendet werden.

Das würde für ein Mappenfeld mit dem Namen `erpId` wie folgt aussehen:

`<default>%FILE_FIELD:erpId%</default>`

Jetzt wird der Wert des Mappenfeldes `erpId` als Ausgangswert für das Gentable – Feld gesetzt (ab DOCUMENTS OS).

`<editable></editable>`: Bestimmt, ob die Tabellenspalte bearbeitet werden kann (`true`) oder nicht (`false`). Die Einstellung greift in der Regel erst nach dem Speichern einer Mappe.

`<maxLength></maxLength>`*:* Gibt die maximale Anzahl von Zeichen für Felder vom Typ TEXT an. Für Felder anderer Datentypen hat die Einstellung keine Funktion.

`<width></width>`: Breite der jeweiligen Tabellenspalte in Pixeln. Standardmäßig richtet sich die Breite der Spalte nach der Spaltenüberschrift.

`<maxWidth></maxWidth>`: Maximale Breite der jeweiligen Tabellenspalte beim Vergrößern in Pixeln.

`<class></class>`: Name einer zusätzlichen CSS Klasse für die Zellen der Spalte. In der Regel liegt eine solche CSS – Datei in einer externen Ressource. Diese Datei muss in der Datei script.jsp als Stylesheet deklariert werden. In der Datei documents.xml muss bei Bedarf ein neuer Pfad für css- Dateien hinterlegt werden.

`<event type="">`
`</event>`: Registriert ein bestimmtes Ereignis (Event) mit dem Eingabeelement der Spalte. Als `type` wird hier der Name des Events verwendet (z.B. `onFocus, onBlur, onChange, onClick, onDblClick)`

Die einzelnen Events werden in der folgenden Art und Weise aufgerufen:

`onChange`: Eingabe neuer Werte und Verlassen der GenTable – Spalte

`onBlur`: Verlassen der GenTable – Spalte

`onFocus`: Eintritt in eine Spalte bzw. Fokussieren der Spalte

`onClick`: Event wird bei Click auf ein Feld (in der Regel eine Spalte vom Typ „Button“) ausgelöst.

`onDblClick`: Event wird beim Doppel- Click auf ein Feld (in der Regel eine Spalte vom Typ „Button“) ausgelöst.

Das Element `event` muss den Namen einer GenTable- Callback-Funktion enthalten, die beim Eintritt des Events ausgeführt werden soll.

Beispiel für eine GenTable – Spalte vom Typ „Button“ mit einem Doppelclick – Event:

```
<field number="12">
        <buttonLabel>ButtonFocusIn</buttonLabel>
        <title>ButtonFocusIn</title>
        <type>BUTTON</type>
        <width>150</width>
        <editable>true</editable>
        <event type="onDblClick">showTabledata</event>
  </field>
```

`<numberFormat>n.2</numberFormat>`: (Default: undefiniert) bestimmt die Anzahl der angezeigten Nachkommastellen (n.0: Keine Nachkommastellen, n.2: Zwei Nachkommastellen).

`<option value="”></option>`: (darf mehrmals vorkommen). Definiert die Optionen, wenn die Spalte vom Typ **SELECT** ist. Im Attribut `value` kann ein technischer Wert angegeben werden, der gespeichert wird, wenn die Option ausgewählt ist. Der Bezeichner zwischen den Tags kann internationalisiert werden. (siehe Kap. 2.6). Alternativ kann der `value` auch mit Semikolon abgetrennt werden: Bsp.:

`<option>green;de:grün;en:green</option>`

Über die pf:Schlüssel – Notation können sprachbezogene Bezeichner abgerufen werden, die in einer Sprachdatei liegen (vgl. Kapitel 2.4.2 `<languagePropertiesFile>` - Tag.)

Mit dem Attribut `description` eines `<option>` - Tags kann ein Tooltip definiert werden.

```
  <field number="8">
        <label>de:Farbe;en:colour</label>
        <title>colour</title>
        <type>SELECT</type>
        <width>70</width>
        <maxLength>10</maxLength>
        <editable>true</editable>
<option description="pf:unitprice">meinWert;pf:unitprice</option>
        <option value="green">de:grün;en:green</option>
        <option value="red">de:rot;en:red</option>
        <option value="blue">de:blau;en:blue</option>
        <option value="yellow">de:gelb;en:yellow</option>
  </field>
```

`<clearExisting></clearExisting>`: Falls bei einer Tabellenspalte externe dynamische Daten in Klapplisten (**SELECT**) integriert werden, aber der aktuell gespeicherte Wert nicht mehr in der Datenbankabfrage vorhanden ist, wird dieser Eintrag aus der Klappliste entfernt (`true`). Ansonsten wird dieser Eintrag zusätzlich mit in die Auswahl genommen (`false, default`). Damit der Wert entfernt wird bzw. hinzugefügt wird nach einer Änderung in der Datenbank, muss das **SELECT** – Feld im Bearbeitungsmodus ausgewählt werden.

`<buttonLabel></buttonLabel>`: Enthält die Aufschrift des Buttons, wenn die Spalte vom Typ **BUTTON** ist.

`<img></img>`: Um herkömmliche Buttons als Elementtyp der Spalte zu grafischen Image-Buttons zu machen, kann hier eine Bilddatei (Dateiname mit Endung .png oder .gif) oder ein Entypo Font-Icon (Notation: „entypo:<Iconname>“) angegeben werden. Ab Documents 5.0 f funktioniert dies auch für Ionicons und Material Design Icons (´Beispiel- Notation: ionicons:ion-md-folder, mdi:mdi-folder). Bitte beachten Sie, dass die Bilddateien nur die Größe des Buttons haben dürfen, da diese nicht automatisch auf deren Größe verkleinert werden. Bei Entypo findet die Skalierung automatisch statt.

Bei Bilddateien muss mit den Attributen width und height die Größe des Bildes festgelegt werden. Die Angabe einer Bilddatei mit Breite und Höhe sowie der Angabe eines Dateipfades in einer externen Ressource müsste daher wie folgt aussehen: (Bsp: `<img width="20" height="20">./ext/pic/bild.png</img>`). In der Datei documents.xml muss bei Bedarf ein neuer Pfad für Bilddateien hinterlegt werden.

`<tooltip></tooltip>`: Tooltip zur Anzeige beim Fahren mit der Maus (Hover) über der Spalte oder einem Feld vom Typ **BUTTON**.

`<group> </group>`: Legt mit `true` oder `false` fest, ob Zeilen mit gleichem Wert für dieses Feld gruppiert werden sollen.

2.4.4 Definition von Toolbar- Buttons

Mit dem Element `<button></button>` können **optional** ein oder mehrere benutzerdefinierte Buttons erstellt werden.

Die Buttons können jeweils über folgende Unterelemente eingestellt werden.

`<label></label>`: Aufschrift des Buttons. (Kann internationalisiert werden, siehe Abschnitt 2.6.2).

`<tooltip></tooltip>`: Tooltip zur Anzeige beim Fahren mit der Maus (Hover) über dem Button.

`<function></function>`: Wenn der Button gedrückt wird, wird die hier definierte SDK – Funktion aufgerufen. Bitte beachten Sie, dass die Funktion erst mittels eines Callbacks registriert werden muss (vgl. Kap.: 3.2) Die Datei, die den Callback beinhaltet, muss zuvor beispielsweise über die Datei script.jsp freigeschaltet worden sein. Bitte tragen Sie Funktionen immer ohne Parameter ein, auch wenn deren Signaturen Parameter (documentsContext, options) enthalten (vgl. Kap.: 3.2.1)!

`<accessKey></accessKey>`: Enthält einen Shortcut. Durch Drücken der ALT-Taste zusammen mit diesem Buchstaben kann die Funktion des Buttons über die Tastatur ausgelöst werden.

`<img></img>`: Einträge analog zu Kapitel 2.4.3.

2.4.5 Die Definition eines Skriptes für Autocomplete

Autocomplete oder *Autovervollständigung* ist eine Funktionalität, die die Arbeit mit *DOCUMENTS* erleichtern soll. Die Eingabe von Feldwerten oder die Suche wird durch *Autocomplete* erleichtert. *Autocomplete* bedeutet, dass bei der Eingabe von Werten zum Beispiel in einem Mappenfeld Vorschlagswerte angezeigt werden, die zu der Eingabe passen und diese sinnvoll vervollständigen würden. Bei der Auswahl eines Vorschlagswertes, wird entweder das gerade aktive Eingabefeld mit Vorschlagswerten gefüllt oder zusätzlich weitere Eingabefelder. Damit Autocomplete funktioniert, muss es in der Konfigurationsdatei für GenTable deklariert werden. Hier wird u.a. ein Skript definiert, das die Vorschlagswerte für Autocomplete zurückgibt. Es gibt unterschiedliche Rückgabeformate für Autocomplete, die im Folgenden erläutert werden sollen:

Json – Formate (Formate in json sind von uns empfohlen):

Format 1 Bei diesem Format werden die ausgewählten Werte auch gleichzeitig in das Auswahlfeld geschrieben

[Wert1, Wert2, …]

Bsp.:

```
var result = [ 'Willi Schreiber', 'Eva Frisch' ];
```

Format 2 Bei diesem JSON – Format werden die ausgewählten Werte auch gleichzeitig in das Auswahlfeld geschrieben (einfache JSON – Form): (Alternative zu Format 1)

[{ value: Wert1}, {value: Wert2}, …]

Bsp.:

```
var result = [
                { value: 'Willi Schreiber' },
                { value: 'Eva Frisch' }
        ];
```

Format 3 Hier können verschiedene Felder gleichzeitig belegt werden. Bei Auswahl von "Willi Schreiber" wird das GenTable – Feld "Abteilung" mit dem Wert "Personalwesen" belegt und das Feld "Aufgaben" mit "Schulung…". (erweiterte JSON – Form):

[{value: <Wert Autocompletefeld>,

fields: [{name: Feld1, value: Wert1}, {name: Feld2…}]

}]

Bsp.:

```
var result = [
                {
                value: 'Willi Schreiber',
                fields: [
                        { name: 'Abteilung', value: 'Personalwesen' },
                        { name: 'Aufgaben', value: 'Schulung / Beratung Konzeption, Evaluierung, Workshop' }
                        ]
                },
                {
                value: 'Eva Frisch',
                fields: [ { name: 'Abteilung', value: 'Controlling' } ]
                }
        ];
```

Rückgabe der oben definierten Variablen:

```
return JSON.stringify(result);
```

Einfache Rückgabe als String: mehrere Felder mittels String-Literal befüllen: Bei Auswahl von "Willi - Schreiber - Personalwesen" werden die Felder „Abteilung" und „Aufgaben" gefüllt:

<Wert1 Autocompletefeld> + "\tFeld1:" + Wert1 + "\tFeld2:" + Wert2… +"\n" + <Wert2 Autocompletefeld> + "\tFeld1…"

Bsp.:

```
var retval = "";
retval += "Willi - Schreiber - Personalwesen" +  "\tAbteilung:" + "Personalwesen" + "\tAufgaben:" + "Schulung / Beratung Konzeption, Evaluierung, Workshop" +  "\n";
retval += "Eva - Frisch - Controlling" +  "\tAbteilung:" + "Controlling" + "\tAufgaben:" + "Kostenträgerrechnung, Kostenstellenrechnung" +  "\n";
return retval;
```

Die wichtigsten Parameter zur Deklaration von Autocomplete sind die folgenden:

Mit dem Element `<autoComplete></autoComplete>` können bei Feldern vom Typ TEXT im Bearbeitungsmodus Vorschlagswerte angezeigt werden.

Hier muss mit `<scriptName></scriptName>` ein Portalskript angegeben werden, welches die Vorschläge zurückgibt.

Für die Autovervollständigung gibt es noch weitere, optionale, Parameter. Diese müssen innerhalb des AutoComplete – Elements eingetragen werden:

`<minQueryChars></minQueryChars>`: Anzahl an Zeichen die eingegeben werden müssen, damit das Autovervollständigungsskript ausgeführt wird (Default: 2).

`<queryDelay></queryDelay>`: Verzögerungszeit nach Eingabe, bis das Skript aufgerufen wird (Default: 0,5 s).

`<maxResults></maxResults>`: Maximale Anzahl an Vorschlägen die angezeigt werden (Default: 50).

`<autoFocusResult></autoFocusResult>`: Fokussiert das erste Element der Vorschlagsliste bei der Einstellung „true“ (Default: false).

`<scriptParam name="Parametername">Wert</scriptParam>`: Skriptparameter die an das Autovervollständigungsskript weitergegeben werden. Diese können im Skript durch Aufruf des Parameternamens referenziert werden.

Folgende Parameterwerte sind möglich (Beispiele dienen der Veranschaulichung):

- Ein fester Wert wird übergeben (Bsp.: „Unternehmensname“).

```
<scriptParam name="Parameter1">Unternehmensname</scriptParam>
```

- Das Mappenfeld mit dem technischen Namen „erpDocumentNumber“ wird ausgelesen.

```
<scriptParam
name="Parameter2">%FILE_FIELD:erpDocumentNumber%</scriptParam>
```

- Das GenTable – Feld mit dem technischen Namen „itemno“ wird ausgelesen und dessen Wert zurückgegeben.

```
<scriptParam name="Parameter3">%FIELD:itemno%</scriptParam>
```

- Der Autotext %userLogin% wird ausgegeben: Andere Autotexte als %userLogin% lassen sich zurzeit nicht aufrufen.

```
<scriptParam name="Parameter4">%userLogin%</scriptParam>
```

Beispielfeld in der Konfigurationsdatei mit Autovervollständigung:

Beispiel für eine Spaltendefinition mit Autocomplete (nur Auszug für Autocomplete ist dargestellt):

```
<field number="1">
    ...
    <autoComplete>
        <scriptName>genAutoComplete</scriptName>
        <minQueryChars>2</minQueryChars>
        <queryDelay>0.5</queryDelay>
        <maxResults>10</maxResults>
            <autoFocusResult>true</autoFocusResult>:
            <scriptParam name="Parameter1">ein fester Wert</scriptParam>
            <scriptParam name="Parameter2">%FILE_FIELD:erpDocumentNumber%</scriptParam>
            <scriptParam name="Parameter3">%FIELD:itemno%</scriptParam>
            <scriptParam name="Parameter4">%userLogin%</scriptParam>
        </autoComplete>
</field>
```

2.4.6 Die Definition eines Skriptes zum Befüllen eines SELECT – Feldes

Mit Hilfe von `<script></script>` kann ein Skript definiert werden, das zum Beispiel zum Befüllen eines SELECT – Feldes genutzt werden kann. Es hat dabei auch Zugriff auf Mappen- und GenTable- Zeilenfelder sowie auf Autotexte.

Mit `<valueDelimiter>;</valueDelimiter>` wird das Semikolon als Trennelement aus dem Skript definiert. So ist es möglich, die einzelnen Elemente eines Rückgabestrings voneinander zu trennen und einzeln in der Klappliste erscheinen zu lassen.

Mit `<keyDelimiter>#</keyDelimiter>` wird das Hash-Symbol als Trennelement aus dem Skript definiert. So ist es möglich, den Speicherwert vom angezeigten Bezeichner des Klapplisteneintrags zu trennen.

Mit `<scriptName>` wird auf das Skript zur Rückgabe der Werte Bezug genommen.

Skript, das ein Stringliteral zurückgibt. Das Semikolon dient zur Trennung der einzelnen Elemente

```
var color = "red#Rot;"
+"yellow#Gelb;"
+ "green#Grün;"
+ "blue#Blau";
```

In der Definitionsdatei werden Parameter definiert. Diese können über ihren technischen Namen im Skript referenziert werden:

```
util.out("Parameter 1: "+ Parameter1);
util.out("Parameter 2: "+ Parameter2);
util.out("Parameter 3: "+ Parameter3);
util.out("Parameter 4: "+ Parameter4);
return color;
```

Eintrag in der Definitionsdatei:

Über die Parameter besteht ein Zugriff auf Mappen- und Zeilenfelder sowie auf Autotexte. Zur näheren Erläuterung der Parametertypen lesen Sie bitte Kapitel 2.4.5.

```
<field number="1">
        <label>Farbe</label>
        <title>selectBeispielFarbe</title>
        <type>SELECT</type>
        <width>90</width>
        <editable>true</editable>
        <script>
                <scriptName>stringGentable</scriptName>
                <delimiter>;</delimiter>
                <scriptParam name="Parameter1">Unternehmensname</scriptParam>
                <scriptParam name="Parameter2">%FILE_FIELD:erpOrderNo%</scriptParam>
                <scriptParam name="Parameter3">%FIELD:itemno%</scriptParam>
                <scriptParam name="Parameter4">%userLogin%</scriptParam>
        </script>
</field>
```

2.4.7 Einschränkungen des Inhalts und der Sichtbarkeit

2.4.7.1 Einschränkungen des Inhalts

Für ein Feld können verschiedene Einschränkungen eingestellt werden:

Für ein Textfeld mit numerischer Eingabe muss ein Constraint `<constraint>NUMBER</constraint>` hinzugefügt werden. Das Feld wird dann beim Speichern auf die korrekte Eingabe überprüft.

Wenn das Feld als Pflichtfeld angelegt werden soll, kann `<constraint>MANDATORY</constraint>` für dieses Feld angegeben werden.

2.4.7.2 Einschränkungen der Sichtbarkeit

Es ist möglich, die Sichtbarkeit von Zeilen und Spalten im GenTable-Plugin abhängig von verschiedenen Bedingungen zu steuern.

- Es gibt zwei Regeltypen „`READONLY`“ (schreibgeschützt) und „`INVISIBLE`“ (nicht sichtbar)
- Wird in einer Regel das Attribut „`invert`“ auf „`true`“ gesetzt, wird die Logik der Regel umgekehrt.
- Eine „rowCondition“ legt eine Sichtbarkeitsregel mit oben genannten Attributen für eine Zeile fest, wohingegen eine „condition“ ein ebensolche Regel für eine Spalte festlegt.
- Die grundlegende Sichtbarkeitsregel ohne weitere Regeln sieht wie folgt aus:

```
<condition>
      <rule type="INVISIBLE"/>
</condition>
```

Wenn eine condition mit <rule type="INVISIBLE" /> definiert ist, dann sollen die Inhalte dieser Spalte ausgeblendet werden.[1] – Diese Regel existiert nur als „condition“ und nicht als „rowCondition“.

Wichtiger Hinweis: Die Einschränkung der Sichtbarkeit hat lediglich Auswirkungen in der Web-Oberfläche. Das zugrunde liegende Datenmodell steht dem Browser grundsätzlich vollständig zur Verfügung, so dass die Einschränkung der Sichtbarkeit nicht als eine Einschränkung der Benutzerrechte verstanden werden darf.

Darüber hinaus gibt es folgende Regeltypen (jeweils mit Beispielen):

a) Ein Feld einer Zeile mit einem festen Wert vergleichen.

Bei der folgenden rowCondition wird die Zeile „READONLY“, wenn das GenTable - Feld „itemtext“ den Wert „readonlyrow“ hat. – Diese Anweisung lässt sich auch invertieren.

```
<rowCondition>
      <rule type="READONLY" field="itemtext" value="readonlyrow"/>
</rowCondition>
```

Invertierte Regel: Die Zeile ist jetzt nur beschreibbar, wenn im GenTable – Feld „itemtext” der Wert „readonlyrow“ steht. In allen anderen Fällen ist die Zeile nicht beschreibbar sondern „READONLY“.

```
<rowCondition>
      <rule type="READONLY" field="itemtext" value="readonlyrow" invert="true"/>
</rowCondition>
```

b) Ein Feld einer Zeile mit einem festen Wert vergleichen und prüfen, ob der aktuelle Benutzer Mitglied eines bestimmten Zugriffsprofils ist.

Im vorliegenden Beispiel ist die Zeile „readonly“, wenn im Feld „itemtext“ der Wert „readonlyme“ steht und der ausführende Benutzer zum Zugriffsprofil „Profilname“ gehört. Beide Bedingungen müssen zutreffen.

[1] Zusätzlich gibt es hier den Schalter <columnsAlwaysVisible>true|false</columnsAlwaysVisible>, der standardmäßig auf „true“ gesetzt sein soll. Bei columnsAlwaysVisible mit dem Wert „true“ und condition mit dem Wert „INVISIBLE“, wird der Spaltentitel trotzdem angezeigt. Mit columnsAlwaysVisible mit dem Wert „false“ und condition mit dem Wert „INVISIBLE“, wird der Spaltentitel ausgeblendet.

```
<rowCondition>
      <rule type="READONLY" field="itemtext" value="readonlyme"
      accessprofile="Profilname"/>
</rowCondition>
```

c) Prüfen, ob der aktuelle Benutzer Mitglied eines bestimmten Zugriffsprofils ist

Im hier dargestellten Fall ist die erzeugte Zeile nach dem Speichern nicht mehr sichtbar für das Zugriffsprofil „hideall“. Für Benutzer außerhalb des Profils sind die Eingaben sichtbar.

```
<rowCondition>
      <rule type="INVISIBLE" accessprofile="hideall"/>
</rowCondition>
```

d) Ein Mappenfeld mit einem festen Wert vergleichen:

Im Falle, dass im Mappenfeld „Vendor“ „hideItemText“ steht, wird die GenTable – Spalte „invisible“.

```
<condition>
       <rule type="INVISIBLE" filefield="Vendor" value="hideItemText" />
</condition>
```

e) Ein Mappenfeld mit einem festen Wert vergleichen und prüfen, ob der aktuelle Benutzer Mitglied eines bestimmten Zugriffsprofils ist:

Wenn im Mappenfeld „Vendor“ „hidemecell“ steht und der Benutzer Mitglied des Zugriffsprofils „hidecells“ ist, wird die Spalte ausgeblendet.

```
<condition>
      <rule type="INVISIBLE" filefield="Vendor" value="hidemecell"
      accessprofile="hidecells"/>
</condition>
```

f) Ein Mappenfeld mit dem Wert eines Feldes einer Zeile vergleichen

Hier wird der Inhalt des GenTable – Feldes „itemtext“ mit dem Inhalt des Mappenfeldes „Vendor“ verglichen. Wenn die Werte gleich sind, wird die Zeile ausgeblendet. An den Wert aus dem Mappenfeld gelangt man hier mittels einer speziellen Syntax. Sie ähnelt den aus DOCUMENTS bekannten Autotexten, die hier jedoch nicht verwendet werden können.

```
<rowCondition>
       <rule type="INVISIBLE" field="itemtext" autotext="file:%Vendor%"/>
</rowCondition>
```

g) Ein Mappenfeld mit dem Wert eines Feldes einer Zeile vergleichen und prüfen, ob der aktuelle Benutzer Mitglied eines bestimmten Zugriffsprofils ist.

Wenn die Werte im GenTable – Feld „itemtext“ und im Mappenfeld „vendor“ übereinstimmen und der aktuelle Benutzer zum Profil „readonly“ gehört, wird die Zeile ausgeblendet.

```
<rowCondition>
      <rule type="INVISIBLE" field="itemtext" autotext="file:%Vendor%"
      accessprofile="readonly"/>
</rowCondition>
```

h) Ein Mappenfeld und/oder ein Feld einer Zeile mit dem Wert eines systemweiten Autotexts oder einer Eigenschaft des aktuellen Benutzers vergleichen. Hier wird ein Feld einer GenTable – Zeile mit dem Wert der Eigenschaft „hideGentable“ am Benutzer verglichen. Wenn die Werte gleich sind, so wird die Zeile ausgeblendet.

```
<rowCondition>
      <rule type="INVISIBLE" field="itemtext"
      autotext="%currentUser.$hideGentable%"/>
</rowCondition>
```

Hier wird das Mappenfeld „Vendor“ mit dem Wert der Eigenschaft „hideGentable“ am Benutzer verglichen. Wenn die Werte gleich sind, so wird die Zeile ausgeblendet.

```
<rowCondition>
      <rule type="INVISIBLE" filefield="Vendor"
      autotext="%currentUser.$hideGentable%"/>
</rowCondition>
```

2.4.7.3 Beispieldefinitionsdatei

```
<?xml version="1.0" encoding="utf-8"?>
<table_def name="ftInvoice">
  <xmlfield>accountAssignment</xmlfield>
  <lastRowEditable>false</lastRowEditable>

  <rowCondition>
        <rule type="READONLY" field="itemtext" value="readonlyrow" invert="true"/>
        <rule type="READONLY" field="itemtext" value="readonlyme"
               accessprofile="readonly"/>
        <rule type="INVISIBLE" accessprofile="hideall"/>
        <rule type="INVISIBLE" field="itemtext" autotext="file:%Vendor%"/>
        <rule type="INVISIBLE" field="itemtext" autotext="file:%Vendor%"
               accessprofile="readonly"/>
        <rule type="INVISIBLE" field="itemtext"
               .autotext="%currentUser.$hideGentable%"/>
        <rule type="INVISIBLE" filefield="Vendor"
               .autotext="%currentUser.$hideGentable%"/>
  </rowCondition>

  <field number="1">
        <label>de:Artikelnr.;en:item number</label>
        <title>itemno</title>
        <type>TEXT</type>
        <editable>false</editable>
        <width>100</width>
```

```
</field>
<field number="2">
        <label>de:Artikeltext;en:description</label>
        <title>itemtext</title>
        <type>TEXTAREA</type>
        <width>200</width>
        <constraint>MANDATORY</constraint>
        <editable>true</editable>
        <condition>
                <rule type="INVISIBLE" filefield="Vendor" value="hideItemText" />
                <rule type="INVISIBLE" filefield="Vendor" value="hidemecell"
                        .accessprofile="hidecells"/>
        </condition>
</field>

<field number="3">
        <label>de:Menge;en:amount</label>
        <title>amount</title>
        <type>TEXT</type>
        <width>50</width>
        <editable>true</editable>
        <class>number</class>
        <event type="onChange">setPosAmount</event>
        <constraint>NUMBER</constraint>
</field>

<button>
        <label>de:Neue Zeile;en:New row</label>
        <tooltip>de:Neue Zeile hinzufügen;en:Add new Row</tooltip>
        <function>appendNewRow();</function>
</button>

<button>
        <label>de:Zeile(n) kopieren;en:copy row(s)</label>
        <tooltip>de:Ausgewählte Zeile(n) kopieren;en:Copy selected row(s)</tooltip>
        <function>cloneRow();</function>
</button>

<button>
        <label>de:Zeile(n) löschen;en:delete row(s)</label>
        <tooltip>de:Ausgewählte Zeile(n) löschen;en:Delete selected row(s)</tooltip>
        <function>deleteRow();</function>
```

```
  </button>

  <button>
        <label>de:Splitbuchung;en:split entry</label>
        <tooltip>de:Ausgewählte Zeile(n) splitten;en:Split selected row(s)</tooltip>
        <function>splitAccounting();</function>
  </button>

  <button>
        <label>de:Nettobetrag prüfen;en:check after deduction</label>
        <tooltip>de:Nettobetrag aller Zeilen splitten;en:Check all rows after
              deduction</tooltip>
        <function>compareAmount();</function>
  </button>
  <button>
        <label>de:Nach oben;en:up</label>
        <tooltip>de:Zeile(n) nach oben verschieben;en:Move rows up</tooltip>
        <function>moveUpRow();</function>
  </button>
  <button>
        <label>de:Nach unten;en:down</label>
        <tooltip>de:Zeile(n) nach unten verschieben;en:Move rows down</tooltip>
        <function>moveDownRow();</function>
  </button>

</table_def>
```

2.4.8 Abfrage externer Datenbanken

`<sql></sql>`: Kann verwendet werden, um die Inhalte des Feldes aus einer externen Datenquelle zu beziehen. Hierbei wird die am Anfang des Dokuments angegebene Datenbankverbindung genutzt, um die Daten zu beziehen.

Die Abfrage der Datenbank kann für das Feld im Element `<sql></sql>` mit folgenden Unterelementen eingestellt werden:

`<query></query>`: Enthält den SQL-Befehl der zur Abfrage verwendet werden soll. Bei der Angabe des Befehls können verschiedene Platzhalter verwendet werden.

- `%userLogin%` **:** Wird durch den Loginnamen des aktuell angemeldeten Benutzers ersetzt.
- `%FILE_FIELD:Mappenfeldname%` **:** Wird durch den Inhalt des jeweiligen Mappenfeldes der aktuellen Mappe ersetzt.
- `%FIELD:Spaltenname%` **:** Wird durch den Wert der jeweiligen Spalte der aktuellen GenTable- Tabellenzeile ersetzt.

Es ist darauf zu achten, dass alle Platzhalter immer in **Hochkommata** (z.B. `'%FIELD:Artikel%'`) eingeschlossen sind. Externe Daten können in Klapplisten, Textfelder und mehrzeilige Textfelder integriert werden.

`<result></result>`: Enthält den Namen der Datenbankspalte, die für die Abfrage der Daten verwendet werden soll.

`<key></key>`: Enthält den Namen der Datenbankspalte, die den technischen Wert des Ergebnisses enthält. Es handelt sich hierbei um die Spalte mit dem Primärschlüssel. Im unten stehenden Beispiel werden im `<result>` Tag die Ergebnisse aus der Spalte Artikelname zurückgegeben. Außerdem werden über den `<key>` - Tag die Primärschlüssel zu den jeweiligen Ergebnissen der Abfrage zurückgegeben. Nur wenn man zusätzlich `<key>` verwendet, kann man auf dessen Werte per SDK – Funktion eindeutig zugreifen. Es werden bei Verwendung von `<key>` eindeutige Primärschlüssel abgespeichert, denen Werte zugeordnet werden. Beispiel für ein GenTable – Element vom Typ SELECT (Klappliste mit Liste von vordefinierten Einträgen / Aufzählung):

<option value="1">Aufzählung_1</option>

Wenn nur `<result>` ohne Verwendung von `<key>` verwendet wird, so wird in der entstehenden Klappliste auch für den Schlüssel der Wert abgespeichert.

Beispiel:

<option value="Aufzählung_1">Aufzählung_1</option>

Bei doppelten Bezeichnern (Werten) wären auch die Schlüssel identisch.

Beispiel für eine Datenbankabfrage in einem Feld:

```
<field number="2">
        <label>de:Mitarbeiter;en:employee</label>
        <title>itemtext</title>
        <type>SELECT</type>
        <width>200</width>
        <editable>true</editable>
  <sql>
        <query>SELECT DISTINCT Artikelname, ArtikelNr FROM
        [Northwind].[dbo].[Artikel]</query>
        <result>Artikelname</result>
        <key>ArtikelNr</key>
  </sql>
  </field>
```

Anmerkung zu Datenbankverbindungen:

In der Funktion appendNewRow() o.ä. zum Hinzufügen neuer Zeilen in der Datei invoice.js sollte ein row.reload(); integriert sein. Sonst wird beim Hinzufügen neuer Zeilen nur eine Zeile erzeugt und die dahinter liegenden Skripte nicht geladen. – Wenn die Skripte eine Datenbankverbindung aufbauen, würde diese für den Fall eines fehlenden row.reload() nicht aufgebaut (vgl. Kap. 3.2.1).

2.4.9 Alternative Auswahl der Konfigurationseinstellungen

2.4.9.1 Mehrere Konfigurationsdateien für einen Mappentypen

Es ist möglich, für einen Mappentypen die Verwendung von mehreren Definitionsdateien zu ermöglichen. Hierbei kann der Benutzer entscheiden, welche Definitionsdatei für das GenTable-Plugin zu einer bestimmten Zeit bzw. bei einem bestimmten Workflow-Zustand verwendet werden soll. Dies kann über die Eigenschaft „gentableDefField“ eingestellt werden. Der Wert dieser Eigenschaft muss dem technischen Namen eines Auswahlfeldes im Mappentypen entsprechen. Dieses Auswahlfeld muss als Auswahlmöglichkeiten die Namen der verschiedenen zur Verfügung stehenden Definitionsdateien beinhalten. Nach der Auswahl einer Option und dem anschließenden Speichern, wird die neue Definitionsdatei der GenTable – Tabelle zugrunde gelegt. Wenn man möchte, dass die Mappe und somit die GenTable – Tabelle unmittelbar nach Auswahl einer Definition ohne Speichern aktualisiert wird, muss ein Exit des Typs „bei Wertänderung“ mit der Option „bei Ereigniseintritt aktualisieren“ auf dem oben genannten Auswahlfeld hinterlegt sein.

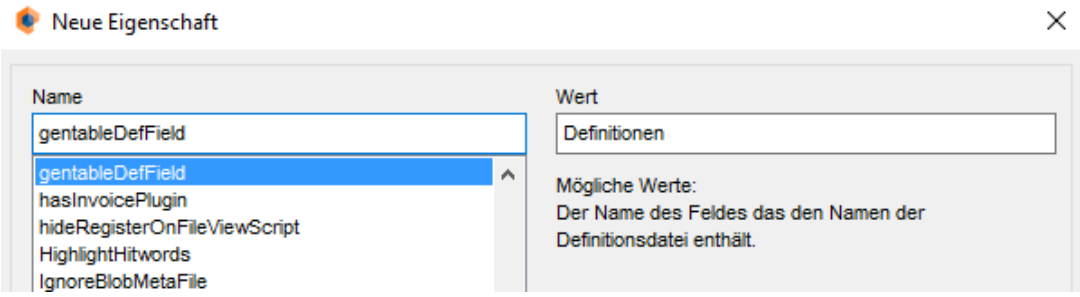


Abb. 10: Eigenschaft gentableDefField

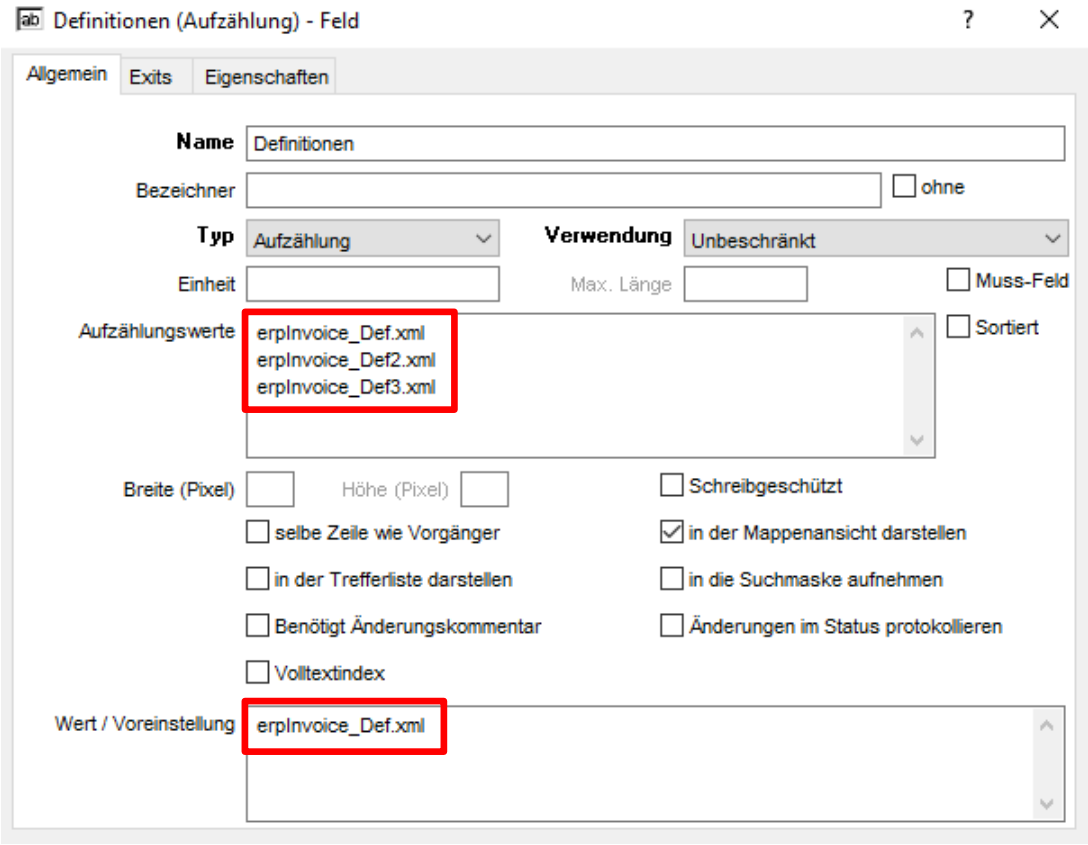


Abb. 11: Auswahlfeld mit Definitionsdateien

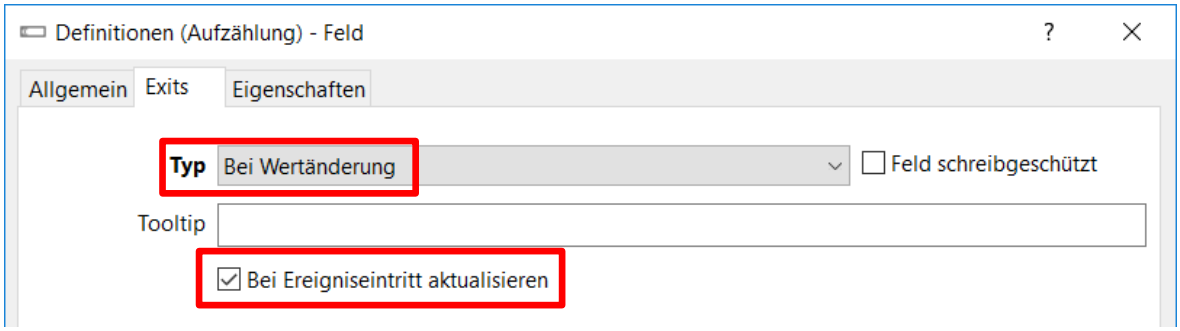


Abb. 12: Feldexit „bei Wertändeurng“ auf dem Auswahlfeld für Definitionsdateien

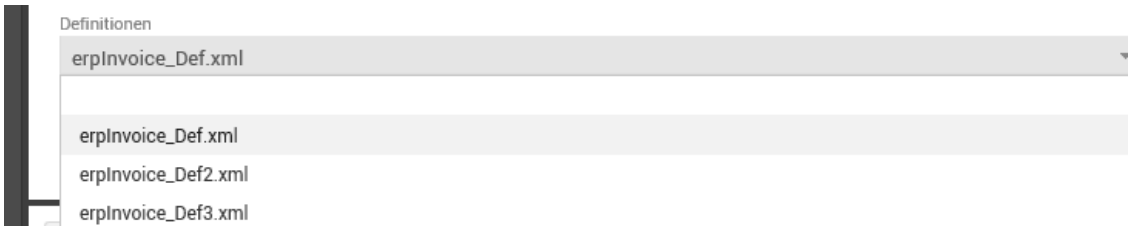


Abb. 13: Aufzählungsfeld zur Auswahl der GenTable - Definitionsdateien

2.4.9.2 Skript zur Rückgabe des Inhalts einer Konfigurationsdatei

Mit Hilfe der Eigenschaft `gentableDefScriptName = Skriptname` am Mappentyp kann ein Portalskript referenziert werden, welches die gesamte GenTable- Definition für diesen Mappentypen als String zurückgibt.

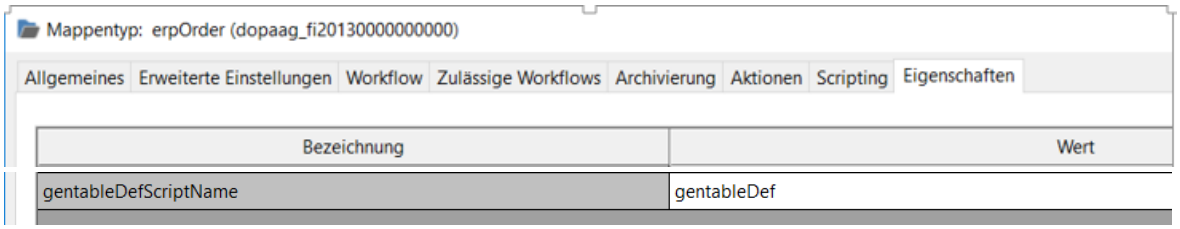


Abb. 14: Einbindung eines Skriptes mit gentableDefScriptName am Mappentyp

```
util.out("Das Skript gentableDef wird aufgerufen.");
var gentableDef ='<?xml version="1.0" encoding="UTF-8"?>\
<table_def name="erpOrder">\
\
    <xmlfield>erpOrderLines</xmlfield>\
    <alwaysShowToolbar>false</alwaysShowToolbar>\
\
    <field number="1">\
        <label>de:Artikelnr.;en:item number</label>\
        <title>itemno</title>\
        <type>TEXT</type>\
        <width>90</width>\
        <editable>true</editable>\
    </field>\
\
```

Abb. 15: Beispiel für den Anfang eines Definitionsskriptes

```
    <button>\
        <label>de:Nach oben;en:up</label>\
        <function>moveUpRow</function>\
    </button>\
    \
    <button>\
        <label>de:Nach unten;en:down</label>\
        <function>moveDownRow</function>\
    </button>\
\
</table_def>';

return gentableDef;
```

Abb. 16: Beispiel für das Ende eines Definitionsskriptes

2.4.9.3 GenTable – Definition als globale Eigenschaft

Eine GenTable Definiton kann in Documents auch als globale Eigenschaft hinterlegt werden.

Hier empfiehlt sich ein Namensschema: `Mappentypname_Def`. Bei Typ muss `gentableDefinition` angegeben werden. Die eigentliche GenTable – Definition unterscheidet sich nicht von derjenigen in einer Definitionsdatei.

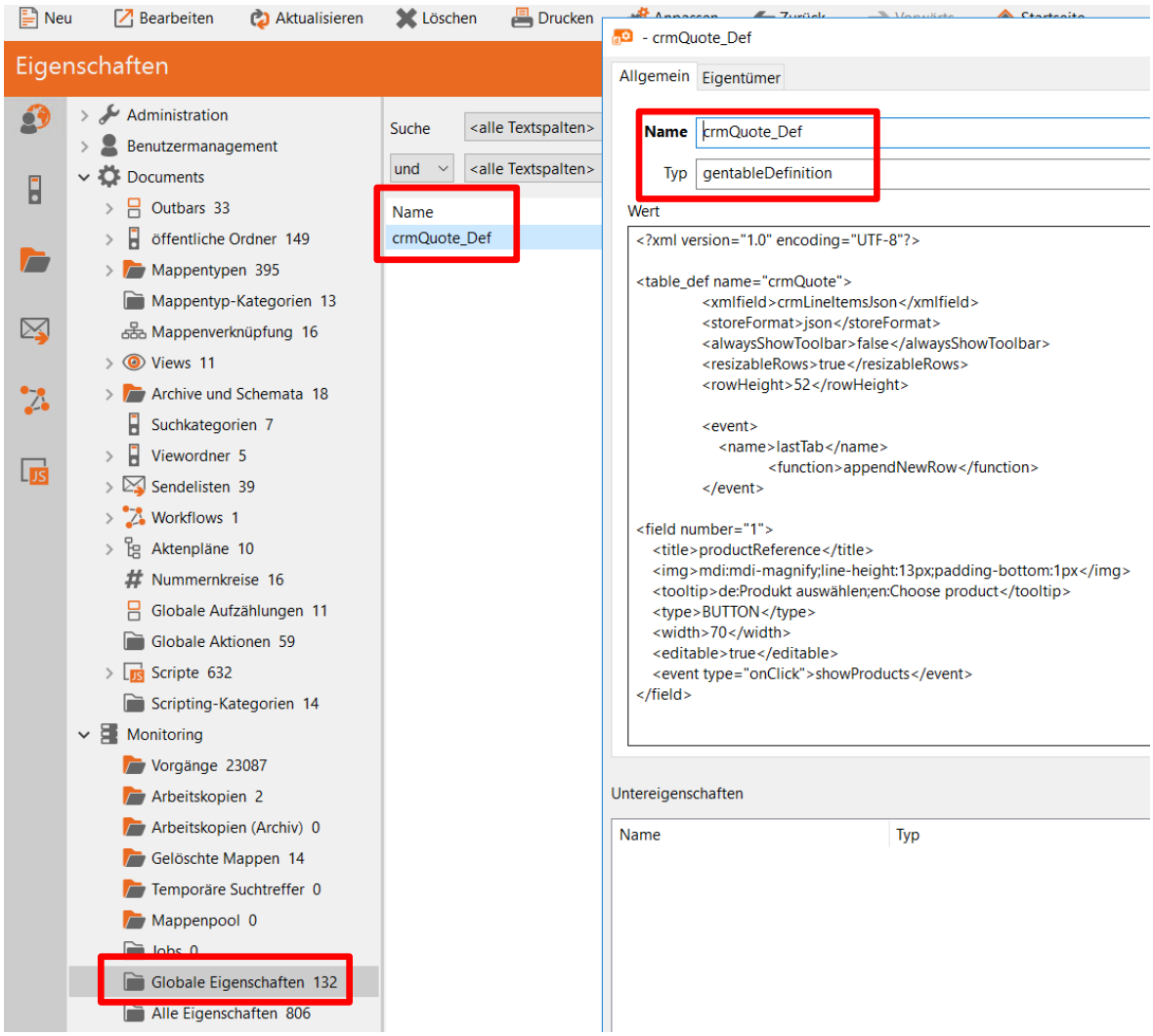


Abb. 17: Beispiel für eine globale Eigenschaft vom Typ gentableDefinition

Die globale Eigenschaft kann beispielsweise in Form eines Skriptes aufgerufen werden. Das Skript ruft im vorliegenden Beispiel die globale Eigenschaft `crmQuote_Def` auf und gibt an, dass diese vom Typ `gentableDefinition` ist.

```
/**
 * Sample script for loading the gentable definition for file type crmQuote
 * Script is configured as file type property
"gentableDefScriptName=crmQuote_gentableDefinitionScript"
 */


// get definition from custom propery
var definitions = context.getCustomProperties("crmQuote_Def", "gentableDefinition"),
    gentableDefinition = definitions.first();


//  return definition if available
if (gentableDefinition) {
    context.returnValue = gentableDefinition.value;
```

```
}


else {
    context.errorMessage = context.getLocaleValue("de:Die Gentable-Konfiguration
konnte nicht geladen werden;en:The Gentable configuration could not be loaded.");
    context.returnValue = -1;
}
```

Das Skript kann durch die Eigenschaft gentableDefScriptName am betreffenden Mappentypen aufgerufen werden:

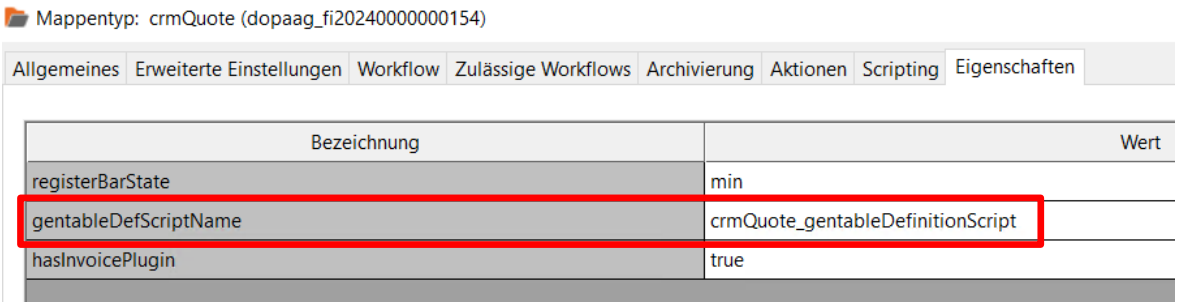


Abb. 18: Aufruf eines Definitionsskriptes für GenTable am Mappentypen

2.5 Anpassung der benutzerdefinierten Skript-Datei

Es gibt die Möglichkeit, eine Skript-Datei bzw. JSP-Datei mit benutzerdefinierten JavaScript-Funktionen in GenTable einzubinden. Im Standardfall ist dies die mitgelieferte Datei `invoice.js`. In dieser Datei sind wichtige Funktionalitäten implementiert wie z.B. das Kopieren von Tabellenzeilen, Löschen von Tabellenzeilen, Anhängen einer neuen Tabellenzeile und Verschieben von Tabellenzeilen. Des Weiteren sind aber auch spezifischere Funktionalitäten für den Einsatz zur Rechnungseingangsprüfung als **Invoice-Plugin** implementiert. Dies sind z.B. Funktionen zum Berechnen der einzelnen Positionspreise, zum Berechnen und Prüfen der Rechnungs-Gesamtsumme, zum Formatieren der Preisfelder und zur Splittbuchung.

2.6 Internationalisierung

GenTable kann mehrsprachig konfiguriert und angezeigt werden. So werden z.B. sämtliche Meldungen der Software-Komponente in der aktuellen Sprache des angemeldeten Benutzers ausgegeben. Vorkonfiguriert sind die Sprachen Deutsch und Englisch. Die einzelnen Meldungen können leicht geändert werden und es können zusätzlich weitere Sprachen hinzugefügt werden.

2.6.1 Sprachdateien

Die originalen Sprachdateien befinden sich im Unterverzeichnis `.../tomcat8/webapps/documents/WEB-INF/classes/` der **DOCUMENTS 5** – Installation. Als Alternative dazu können die Dateien auch als sogenannte externe

Ressourcen in **DOCUMENTS 5** eingebunden werden. Die Sprachdateien tragen alle den Namen `DocumentsMessages_<language>.properties`. Anstelle von `<language>` im Dateinamen steht jeweils das Kürzel der Sprache nach ISO-Norm (z.B. `de` für Deutsch, `en` für Englisch). In diesen `properties`-Dateien sind die einzelnen Sprachmeldungen als Wertepaare definiert. Die Standardsprache ist Deutsch und die zugehörigen Meldungen dafür sind in der Datei `DocumentsMessages_de.properties` hinterlegt. Alle für GenTable relevanten Meldungen der `properties`-Dateien beginnen stets mit dem Bezeichner `Documents.Gentable`.

2.6.2 Konfigurationsdatei

Alle `<label></label>` und `<option></option>` Elemente in der Konfigurationsdatei können internationalisierte Inhalte enthalten. Hierzu wird die folgende Syntax verwendet:

```
<label>Default-Wert;de:Deutsch;en:Englisch;fr:Französisch</label>
```

Der Default-Wert wird im Falle von `<label>` angezeigt wenn die Sprache, die für den aktuellen Benutzer eingestellt ist, nicht in der Beschreibung vorhanden ist.

3 Implementierung eigener Funktionalität

3.1 Einführung

Zur Anpassung von GenTable ist es möglich, weitere selbst implementierte Funktionalität zu integrieren. Dafür werden jedoch grundlegende Programmierkenntnisse in der Sprache JavaScript vorausgesetzt. Die wichtigsten Klassen für die Programmierung mit GenTable stellen der `GentableContext`, das `GentableGridModel`, das `GentableGrid ColumnModel` und das `GentableGridRowModel` dar, welche im weiteren Verlauf des Kapitels detailliert vorgestellt werden.

Das `GentableGridModel` bringt bereits schon einen großen Teil wichtiger Standard-Funktionalitäten und auch auf den Einsatz als **Invoice-Plugin** zugeschnittene Funktionalitäten mit, die dort und in weiteren GenTable-Klassen in unterschiedlichen Funktionen implementiert sind. Eine vollständige Auflistung und Beschreibung aller GenTable-Klassen finden sie zusätzlich in der API Dokumentation für das **DOCUMENTS 5** Client SDK (vgl.: **https://otris.software/documents/api/client-sdk/**).

3.2 Registrierung und Callback-Funktionen

Jegliche selbst implementierte fachliche Funktionalität wird innerhalb sogenannter Callbacks implementiert. Diese Callbacks sind gewöhnliche JavaScript-Funktionen, welche unter einem bestimmten Namen oder für einen bestimmten Zeitpunkt registriert werden.

Die Callbacks können entweder durch das Einbinden einer JavaScript-Datei im externen Kontext oder durch Einbettung im Header-HTML von DOCUMENTS hinzugefügt werden.

https://otris.software/documents/howto/exit/externalResources.html

https://otris.software/documents/howto/common/embed-header-code.html

3.2.1 Callback-Funktionen für Buttons

Um einen Button auf der GenTable Toolbar zu erzeugen, welcher eine selbst programmierte Funktion ausführen soll, sind grundsätzlich zwei Schritte erforderlich. Zunächst muss der Button in der GenTable XML-Konfigurationsdatei definiert werden (siehe 2.4.4) Die Definition beinhaltet neben dem Label (`<label>`) auch den Namen der Callback-Funktion (`<function>`), welche ausgeführt werden soll, sobald der Benutzer den Button betätigt hat. Im folgenden Beispiel ist das für den Button `Neue Zeile` die Funktion `appendNewRow`.

```
<button>
        <label>de:Neue Zeile;en:New row</label>
        <function>appendNewRow</function>
</button>
```

Im zweiten Schritt muss in der JavaScript-Datei, in der die Funktionalität für den Button hinterlegt werden soll, die Callback-Funktion unter dem in der Definition angegebenen Namen registriert und implementiert werden.

Das folgende Code-Beispiel zeigt, wie eine Callback-Funktion für das einfache Hinzufügen einer Zeile für einen Mappentypen `ftInvoice` unter dem Namen `appendNewRow` in der `gentableRegistry` registriert wird. Die eigentliche Implementierung folgt gleich darauf innerhalb des Callbacks `function(documentsContext) { ... }`.

Bitte beachten Sie, dass ein row.reload() hier ergänzt wurde. – Dies führt dazu, dass die Regeln, die in einer Zeile hinterlegt wurden, angewendet werden. Dies bedeutet, dass die Sichtbarkeitsregeln (vgl. Kap. 2.4.7.2) angewandt werden und dass die Datenbankverbindungen über Auswahlfelder zur Verfügung stehen (vgl. Kap. 2.4.8).

```
documents.sdk.gentable.gentableRegistry.registerCallback("ftInvoice",
    "appendNewRow", function(documentsContext) {

var gridModel = documentsContext.getGentableContext().getGridModel(),
      row = gridModel.addRow();
       row.reload();
});
```

3.2.2 GentableContext

Der `GentableContext` vereint sämtliche Funktionen, die im Zusammenhang mit der Implementierung von eigener Funktionalität für GenTable von Bedeutung sind.

Die Funktion `getGridModel()` gibt das `GentableGridModel` der aktuellen Tabelle zurück. Das `GentableGridModel` stellt verschiedene grundlegende Funktionen zur Abfrage und Manipulation von Spalten und Zeilen der Tabelle bereit.

3.2.3 GentableGridModel

Das `GentableGridModel` stellt das komplette clientseitige Datenmodell einer Tabelle in GenTable bereit. Die Spalten und Zeilen dieses Modells werden jeweils durch Objekte der beiden Typen `GentableGridColumnModel` und `GentableGridRowModel` abgebildet, welche vom `GentableGridModel` aus über verschiedene Funktionen direkt erreichbar sind. Es ist grundlegend wichtig zu verstehen, dass sich jegliche Manipulation dieses Datenmodells unmittelbar auf die Anzeige von GenTable auswirkt. Das bedeutet, dass in diesen Fällen entweder die gesamte Tabelle oder nur einzelne Teilbereiche vom Browser automatisch neu gerendert werden, sobald beispielsweise Zeilen hinzugefügt, gelöscht, manipuliert, kopiert oder verschoben werden. Zugriffe und Manipulationen der Tabelle über das browserinterne Document Object Model (DOM) sind somit grundsätzlich nicht mehr erforderlich und entfallen im Vergleich zu früheren Versionen von GenTable komplett. Dadurch wird die Implementierung eigener Funktionalität erheblich vereinfacht. Von jedweder manuellen

Manipulation der GenTable-GUI über dem nativen DOM (z.B. mittels jQuery o.ä.) hingegen wird im Übrigen grundsätzlich abgeraten, da in diesem Fall weder Kompatibilität zu neueren Versionen noch ein einwandfreies Laufzeitverhalten gewährleistet werden kann. Es sollte stets die in diesem Kapitel vorgestellte API verwendet werden.

Um sich in selbst implementierten Callback-Funktionen eine Referenz auf das aktuelle `GentableGridModel` zu beschaffen, sollte zu Beginn stets der nachfolgend abgebildete Code verwendet werden.

```
var gridModel = documentsContext.getGentableContext().getGridModel();
```

Es folgt nun eine Auflistung der wichtigsten Funktionen auf dem `GentableGridModel`. Nähere Informationen entnehmen Sie bitte der Dokumentation zum **Documents** Client SDK und hier den Informationen zum `GentableGridModel`.

Die Funktion `getColumn(name)` gibt das Datenmodell einer beliebigen Spalte aus dem `GentableGridModel` zurück. Zur eindeutigen Bestimmung der Spalte muss der Parameter `name` angegeben werden. Der Rückgabewert ist vom Typ `GentableGridColumnModel`. Auf diesem Modell lassen sich u.a. verschiedene Informationen einer Spalte abfragen.

Die Funktion `getColumns()` gibt sämtliche Spalten auf dem `GentableGridModel` als `GentableGridColumnCollection` zurück.

Die Funktion `columnsSize()` gibt die Anzahl der Spalten auf dem aktuellen `GentableGridModel` als `Integer` zurück.

Die Funktion `getRow(index)` gibt das Datenmodell einer beliebigen Zeile aus dem `GentableGridModel` zurück. Zur eindeutigen Bestimmung der Zeile muss der Parameter `index` angegeben werden. Es ist zu beachten, dass die erste Zeile stets dem Index `0` entspricht. Der Rückgabewert ist vom Typ `GentableGridRowModel`. Auf diesem Modell lassen sich u.a. die einzelnen Zellen einer Zeile lesen und mit neuen Werten besetzen.

Die Funktion `getRows()` gibt sämtliche Zeilen auf dem `GentableGridModel` als `GentableGridRowCollection` zurück.

Die Funktion `rowsSize()` gibt die Anzahl der Zeilen auf dem aktuellen `GentableGridModel` als `Integer` zurück.

Die Funktion `addRow(row)` fügt dem `GentableGridModel` eine neue Zeile hinzu. Der Rückgabewert ist vom Typ `GentableGridRowModel`.

Die Funktion `getSelectedRows()` gibt die Menge aller aktuell über die Checkbox selektierten Zeilen auf dem `GentableGridModel` als `GridCollection` zurück.

Die Funktion `setSelectedRows(rowIndexes)` setzt die Selektion zunächst komplett zurück, so dass keine Zeile mehr selektiert ist. Der Parameter `rowIndexes` bestimmt alle neu zu auszuwählenden Zeilen als `Array` von `Integer`. Es ist zu beachten, dass die erste Zeile stets dem Index `0` entspricht. So würde beispielsweise mit dem Argument `[0, 2]` die erste und die dritte Zeile selektiert.

Die Funktion `resetSelectedRows()` setzt die aktuelle Selektion sämtlicher Zeilen komplett zurück, so dass keine Zeile mehr selektiert ist.

Die Funktion `copyRow(srcIndex, dstIndex)` kopiert eine komplette Zeile. Dabei bestimmen die Parameter `srcIndex` und `dstIndex` den Quell- und Zielindex der zu kopierenden Zeile.

Die Funktion `moveRow(srcIndex, dstIndex)` verschiebt eine komplette Zeile innerhalb des `GentableGridModel`. Dabei bestimmen die Parameter `srcIndex` und `dstIndex` den Quell- und Zielindex der zu verschiebenden Zeile.

Die Funktion `removeRow(index)` entfernt eine komplette Zeile aus dem `GentableGridModel`. Dabei bestimmt der Parameter `index` den Index der zu löschenden Zeile.

3.2.4 GentableGridColumnModel

Das `GentableGridColumnModel` repräsentiert das Datenmodell einer Spalte in GenTable. Da die Definition der einzelnen Spalten in der XML-Konfiguration festgelegt wird, kann an dieser Stelle nur ein lesender Zugriff erfolgen. Es folgt nun eine kurze Auflistung der wichtigsten Funktionen auf dem `GentableGridColumnModel`. Nähere Informationen entnehmen Sie bitte der Dokumentation zum Documents Client SDK und hier den Informationen zum `GentableGridColumnModel`.

Die Funktion `getName()` gibt den technischen Namen der Spalte zurück.

Die Funktion `getLabel()` gibt das sprachenabhängige Label der Spalte zurück.

Die Funktion `getDataType()` gibt den Datentyp der Spalte als `String` zurück.

Die Funktion `isVisible()` gibt `true` zurück, falls die Spalte sichtbar ist, ansonsten `false`.

Die Funktion `isEditable()` gibt `true` zurück, falls die Spalte editierbar ist, ansonsten `false`.

3.2.5 GentableGridRowModel

Das `GentableGridRowModel` repräsentiert das Datenmodell einer Zeile in GenTable. Es folgt nun eine kurze Auflistung der wichtigsten Funktionen auf dem `GentableGrid RowModel`.

Die Funktion `get(columnName)` gibt den Wert einer bestimmten Zelle in der Zeile zurück. Der Parameter `columnName` bestimmt dabei die gewünschte Spalte.

Die Funktion `getNumber(columnName)` gibt den Wert einer bestimmten Zelle als JavaScript-Number- Objekt zurück. Dies hat den Vorteil, dass man mit dem Spaltenwert rechnen kann, ohne ihn konvertieren zu müssen. Der Parameter `columnName` bestimmt dabei die gewünschte Spalte.
Die Funktion `index()` gibt den Index der Zeile zurück. Es ist zu beachten, dass die erste Zeile stets dem Index `0` entspricht.

Die Funktion `isVisible()` gibt den Wert `true` zurück, falls die Zeile sichtbar ist, ansonsten `false`.

Die Funktion `reload()` lädt den Inhalt der Zeile neu und wendet die in der Definitionsdatei hinterlegten Sichtbarkeitsregeln auf die Zeile an. Dynamische datenbankabhängige Werte werden ebenfalls beim `reload()` geladen (vgl. Kap. 2.4.8).

Die Funktion `toJSON()` erzeugt eine Kopie der Zeile im standardisierten `JSON`-Format.

4 Einsatzbeispiele & Standardfunktionen

Im Folgenden werden die am häufigsten verwendeten Funktionen für den Einsatz als **Invoice-Plugin** erläutert. Nach dem Laden einer Mappe des Typs *„erpOrder“* sieht die Anzeige typischerweise so aus:

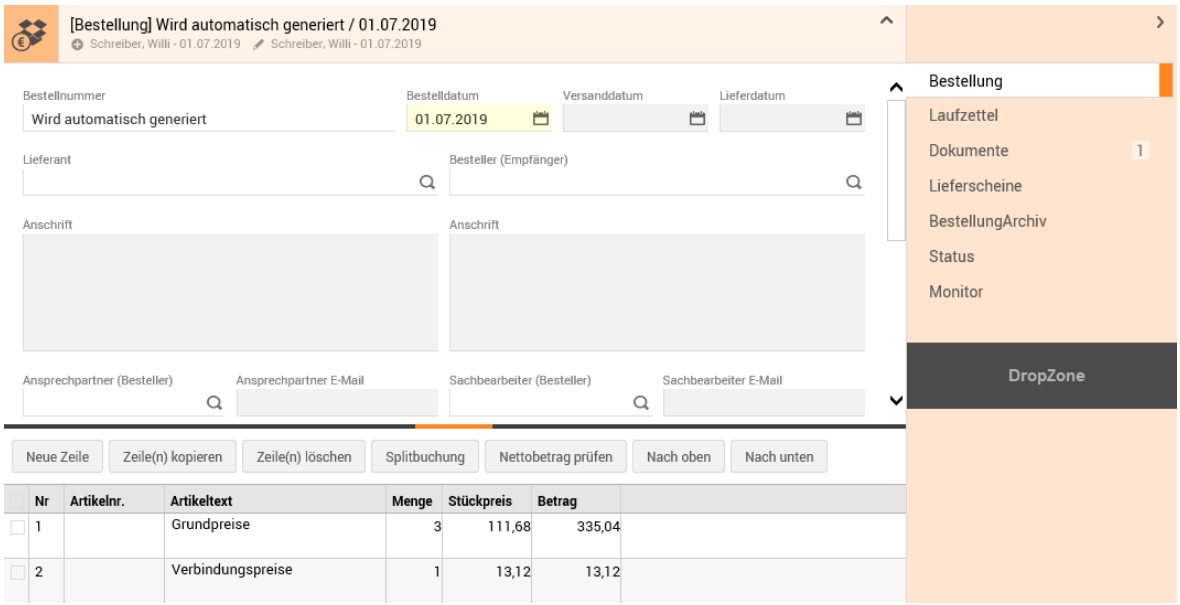


Abb. 19: Mappe mit Invoice-Plugin

In den Feldern der Mappe und in den Rechnungs-Positionen (Tabellenzeilen) wurden einige Beispieleinträge gemacht. In der nachfolgenden Abbildung ist durch den entsprechenden Button eine neue Zeile angefügt worden.

Abb. 20: Neue Zeile anfügen

Neue Zeile | Zeile(n) kopieren | Zeile(n) löschen | Splitbuchung | Nach oben | Nach unten

	Nr	Artikelnr.	Artikeltext	Menge	Stückpreis	Betrag	
☑	1		Grundpreise	3	111,68	335,04	
☐	2		Verbindungspreise	2	13,12	26,24	
☑	3		Vergünstigungen / Guthaben	1	-5,38	-5,38	

Abb. 21: Zeile(n) auswählen

Die zwei über die Index-Checkboxen ausgewählten Zeilen sollen kopiert werden.

Neue Zeile | Zeile(n) kopieren | Zeile(n) löschen | Splitbuchung | Nach oben | Nach unten

	Nr	Artikelnr.	Artikeltext	Menge	Stückpreis	Betrag	
☐	1		Grundpreise	3	111,68	335,04	
☑	2		Grundpreise	3	111,68	335,04	
☐	3		Verbindungspreise	2	13,12	26,24	
☐	4		Vergünstigungen / Guthaben	1	-5,38	-5,38	
☑	5		Vergünstigungen / Guthaben	1	-5,38	-5,38	

Abb. 22: Zeile(n) kopieren

Nach dem Kopieren der Zeilen wird nun eine Zeile ausgewählt und danach gelöscht.

Neue Zeile | Zeile(n) kopieren | Zeile(n) löschen | Splitbuchung | Nach oben | Nach unten

	Nr	Artikelnr.	Artikeltext	Menge	Stückpreis	Betrag	
☐	1		Grundpreise	3	111,68	335,04	
☑	2		Grundpreise	3	111,68	335,04	
☐	3		Verbindungspreise	2	13,12	26,24	
☐	4		Vergünstigungen / Guthaben	1	-5,38	-5,38	
☐	5		Vergünstigungen / Guthaben	1	-5,38	-5,38	

Abb. 23: Zeile(n) löschen

Über die Funktion *„Splitbuchung“* (hier Zeile 2) werden ausgewählte Zeilen gleichzeitig kopiert und die Einträge für Menge und Betrag jeweils halbiert.

Neue Zeile | Zeile(n) kopieren | Zeile(n) löschen | Splitbuchung | Nach oben | Nach unten

	Nr	Artikelnr.	Artikeltext	Menge	Stückpreis	Betrag
☐	1		Grundpreise	3	111,68	335,04
☑	2		Verbindungspreise	2	13,12	26,24
☐	3		Vergünstigungen / Guthaben	1	-5,38	-5,38
☐	4		Vergünstigungen / Guthaben	1	-5,38	-5,38

Abb. 24: vor der Splitbuchung

Neue Zeile | Zeile(n) kopieren | Zeile(n) löschen | Splitbuchung | Nach oben | Nach unten

	Nr	Artikelnr.	Artikeltext	Menge	Stückpreis	Betrag
☐	1		Grundpreise	3	111,68	335,04
☐	2		Verbindungspreise	1	13,12	13,12
☑	3		Verbindungspreise	1	13,12	13,12
☐	4		Vergünstigungen / Guthaben	1	-5,38	-5,38
☐	5		Vergünstigungen / Guthaben	1	-5,38	-5,38

Abb. 25: nach der Splitbuchung

Auch können bestimmte Felder aus den Einträgen anderer Felder automatisch berechnet werden, wie z.B. der Betrag als Produkt aus Menge und Stückpreis.

Beim Prüfen des Nettobetrags erhält man, je nach Ergebnis des Vergleiches mit dem Betrag, der in der Mappe als Feldwert steht eine bestimmte Meldung.

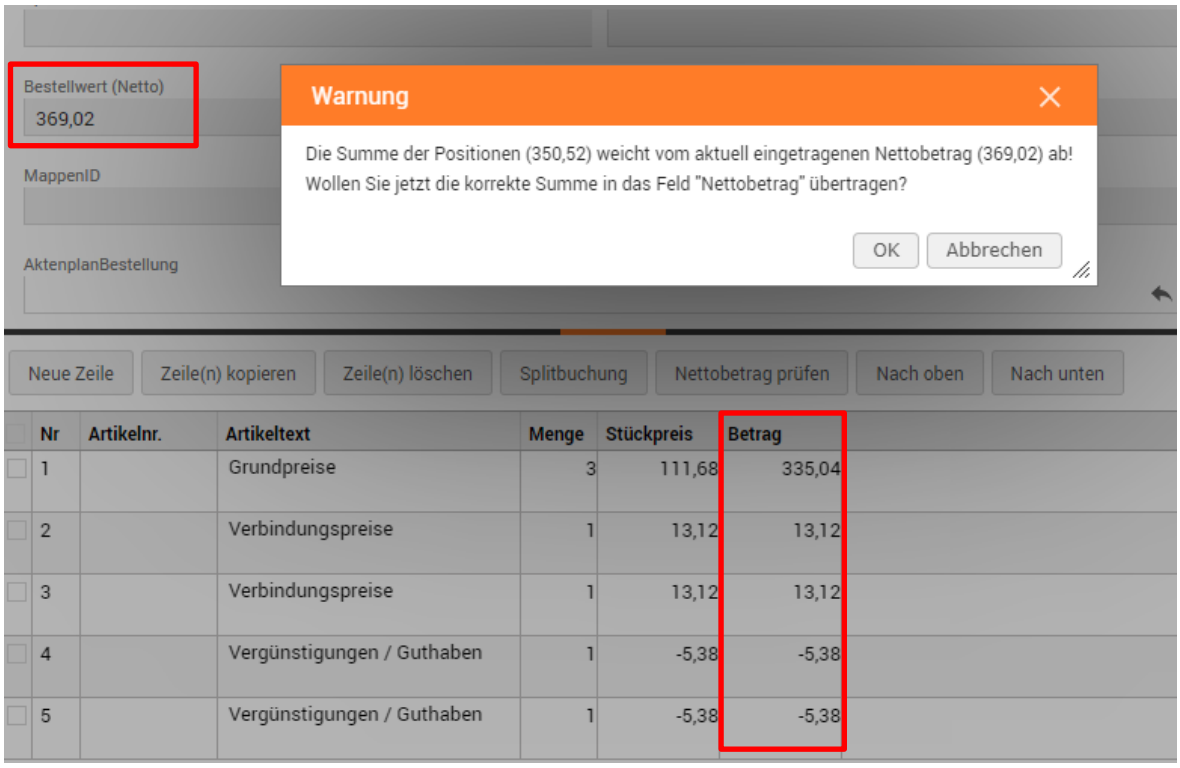


Abb. 26: Nettobetrag prüfen

Wenn bestimmte Tabellenspalten/Felder je nach Konfiguration nur Zahlenwerte enthalten dürfen oder nicht leer bleiben dürfen, erscheint beim Speichern eine entsprechende Meldung. Erst wenn alle Bedingungen erfüllt sind, kann die Tabelle gespeichert werden.

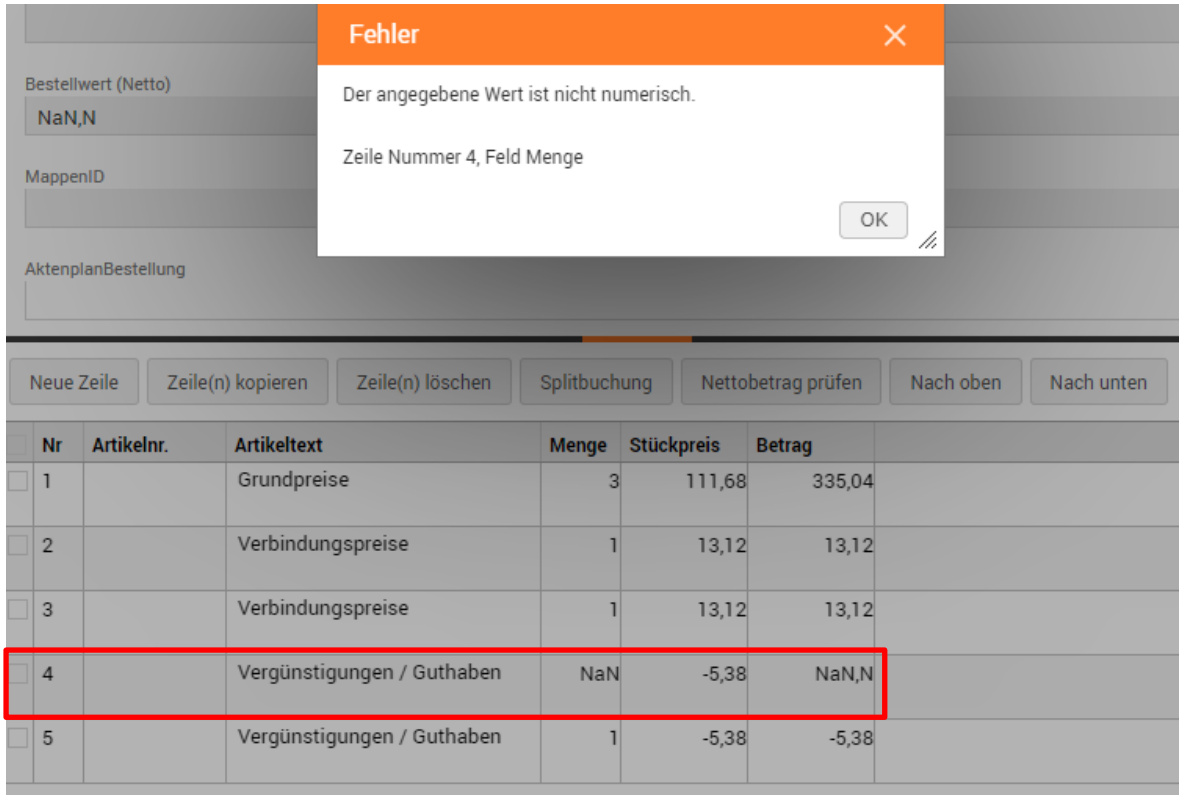


Abb. 27: Überprüfung beim Speichern

5 Troubleshooting: Fehler & Ursachen

Falls keine XML-Konfigurationsdatei für den jeweiligen Mappentyp existiert, der Name falsch geschrieben wurde oder die Datei leer ist, kann GenTable nicht gestartet werden und wird mit folgender Fehlermeldung abgebrochen:

Abb. 28: Fehlende XML-Konfigurationsdatei

Wenn die XML-Konfigurationsdatei nicht korrekt gelesen werden kann, die Datei kein valides XML beinhaltet oder bestimmte, nicht erlaubte Sonderzeichen als XML-Einträge (z.B. „<“, „>“, „&“ etc.) verwendet werden, so wird die Ausführung von GenTable mit folgender Fehlermeldung abgebrochen:

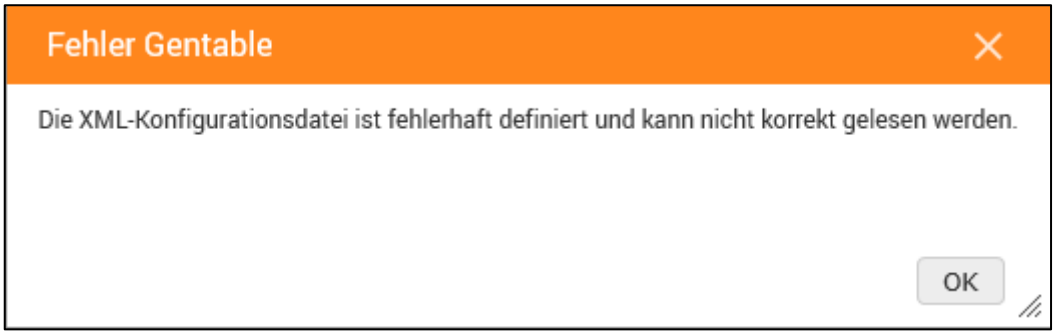


Abb. 29: XML-Konfigurationsdatei fehlerhaft

Der folgende Fehler tritt auf, falls der XML-Eintrag `<xmlfield>` in der XML-Konfigurationsdatei fehlt, falsch definiert oder leer ist.

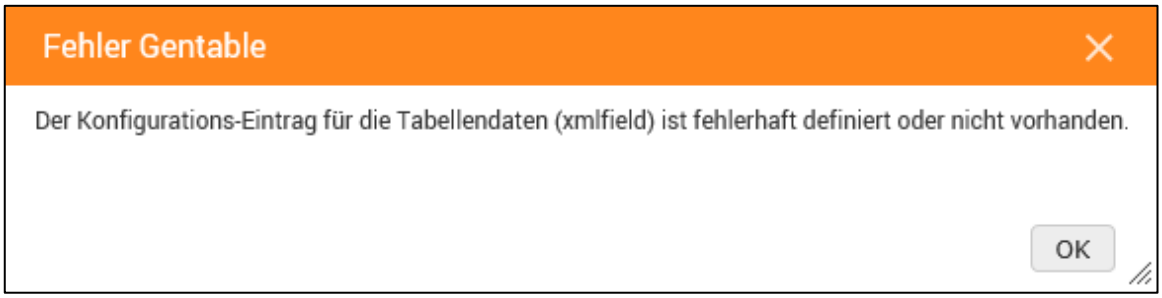


Abb. 30: Konfigurationseintrag für Mappenfeld fehlt

Folgende Fehlermeldung erscheint, wenn das in der XML-Konfigurationsdatei definierte Mappenfeld für die Tabellendaten nicht im Mappentyp selbst definiert bzw. vorhanden ist:

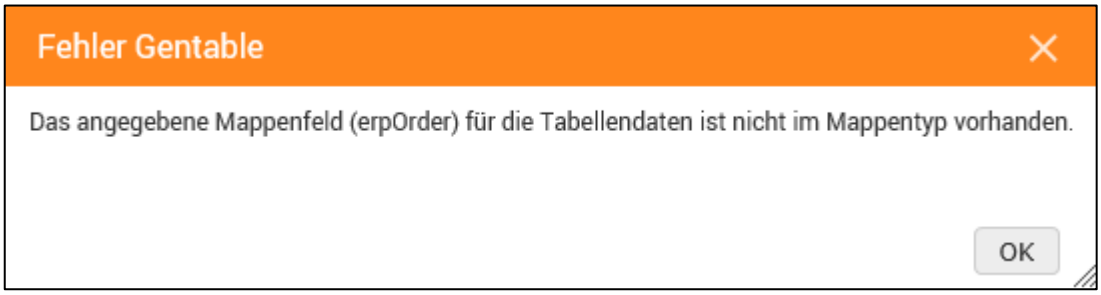


Abb. 31: Mappenfeld nicht vorhanden

Es muss stets mindestens eine Tabellenspalte in der XML-Konfigurationsdatei definiert sein:

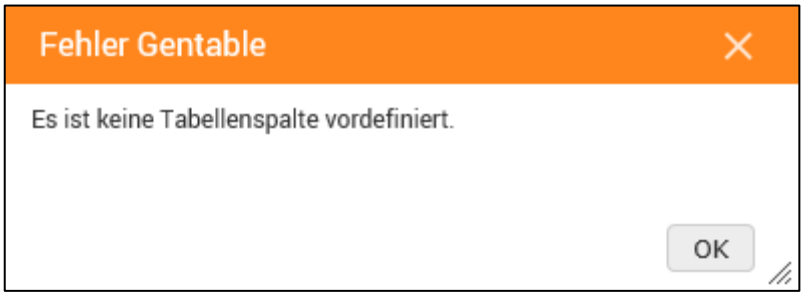


Abb. 32: Keine Tabellenspalten

Falls externe Daten aus einer Datenbank integriert werden sollen, so muss ein passender JDBC-Datenbank-Treiber vorhanden sein.

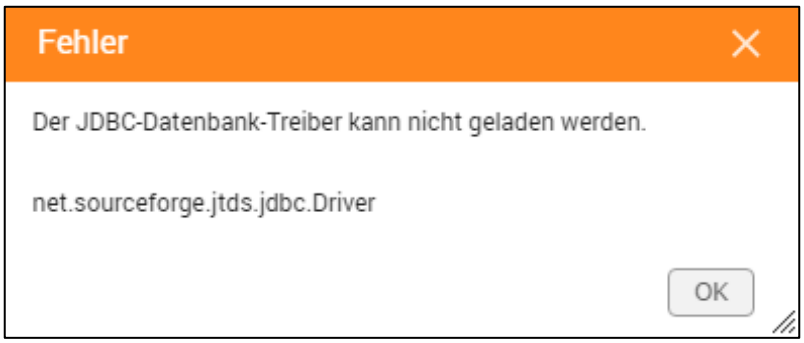


Abb. 33: fehlender Datenbanktreiber

Die SQL-Syntax muss immer korrekt eingehalten werden, ansonsten treten Fehler bei der Datenbank-Anfrage auf. Nähere Hinweise werden in der Fehlermeldung angezeigt:

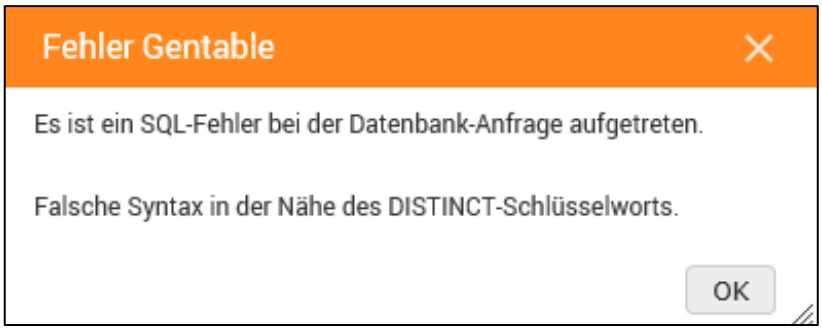


Abb. 34: fehlerhafte Datenbankabfrage

Falls eine in der Definitionsdatei deklarierte Spalte nicht angezeigt wird, kann der Grund sein, dass entweder die bei <field> hinterlegte Nummer der Spalte oder der <title> doppelt vergeben sind.

```
<field number="17">
    <label>de:Betrag:en:total</label>
    <title>total</title>
    <type>NUMBER</type>
    <width>80</width>
    <editable>true</editable>
    <numberFormat>n.2</numberFormat>
</field>

<field number="17">
    <label>de:KST:en:cost unit</label>
    <title>total</title>
    <type>NUMBER</type>
    <width>40</width>
    <maxLength>4</maxLength>
    <editable>true</editable>
    <condition>
        <rule type="READONLY" filefield="
    </condition>
</field>
```

Abb. 35: doppelte Deklarationen von <field> und <title>

Wenn eine Callback – Funktion nicht aufgerufen wird, überprüfen Sie bitte, ob der Name der Funktion in der invoice.js und in der Definitionsdatei übereinstimmen. Für einen Aufruf müssen die Namen exakt übereinstimmen.

Wenn eine Callback – Funktion mehrfach aufgerufen wird, kann die Ursache sein, dass eine Funktion gleichen Namens entweder in derselben Datei oder einer anderen Datei vorliegt. Je nach Anzahl gleichnamiger Funktionen in eingebundenen Dateien wird somit eine Callback – Funktion n – Mal aufgerufen.

6 Abbildungsverzeichnis

Abb. 1: Baumstruktur im DOCUMENTS-Manager 6
Abb. 2: Bearbeiten bzw. Erstellen eines Mappentyps 7
Abb. 3: Eigenschaft hasInvoicePlugin am Mappentyp erpInvoice 7
Abb. 4: Bearbeiten bzw. Erstellen von Feldern 8
Abb. 5: Mappenfeld-Dialog 8
Abb. 6: Die globale Eigenschaft precisionNumeric 10
Abb. 7: Die globalen Eigenschaften groupingNumeric und groupingNumeric_locale 10
Abb. 8: Schema einer GenTable – Konfigurationsdatei 11
Abb. 9: Übersicht der Speicherverfahren 12
Abb. 10: Eigenschaft gentableDefField 30
Abb. 11: Auswahlfeld mit Definitionsdateien 31
Abb. 12: Feldexit „bei Wertändeurng“ auf dem Auswahlfeld für Definitionsdateien 31
Abb. 13: Aufzählungsfeld zur Auswahl der GenTable - Definitionsdateien 31
Abb. 14: Einbindung eines Skriptes mit gentableDefScriptName am Mappentyp 32
Abb. 15: Beispiel für den Anfang eines Definitionsskriptes 32
Abb. 16: Beispiel für das Ende eines Definitionsskriptes 32
Abb. 17: Beispiel für eine globale Eigenschaft vom Typ gentableDefinition 33
Abb. 18: Aufruf eines Definitionsskriptes für GenTable am Mappentypen 34
Abb. 19: Mappe mit Invoice-Plugin 41
Abb. 20: Neue Zeile anfügen 41
Abb. 21: Zeile(n) auswählen 42
Abb. 22: Zeile(n) kopieren 42
Abb. 23: Zeile(n) löschen 42
Abb. 24: vor der Splitbuchung 43
Abb. 25: nach der Splitbuchung 43
Abb. 26: Nettobetrag prüfen 44
Abb. 27: Überprüfung beim Speichern 44
Abb. 28: Fehlende XML-Konfigurationsdatei 45
Abb. 29: XML-Konfigurationsdatei fehlerhaft 45
Abb. 30: Konfigurationseintrag für Mappenfeld fehlt 45
Abb. 31: Mappenfeld nicht vorhanden 46
Abb. 32: Keine Tabellenspalten 46
Abb. 33: fehlender Datenbanktreiber 46
Abb. 34: fehlerhafte Datenbankabfrage 47
Abb. 35: doppelte Deklarationen von <field> und <title> 47